Synchronous Programming with Refinement Types

JIAWEI CHEN, University of Michigan, USA
JOSÉ LUIZ VARGAS DE MENDONÇA, University of Michigan, USA
BEREKET SHIMELS AYELE*, Addis Ababa Institute of Technology, Ethiopia
BEREKET NGUSSIE BEKELE*, Addis Ababa Institute of Technology, Ethiopia
SHAYAN JALILI, University of Michigan, USA
PRANJAL SHARMA, University of Michigan, USA
NICHOLAS WOHLFEIL, University of Michigan, USA
YICHENG ZHANG, University of Michigan, USA
JEAN-BAPTISTE JEANNIN, University of Michigan, USA

Cyber-Physical Systems (CPS) consist of software interacting with the physical world, such as robots, vehicles, and industrial processes. CPS are frequently responsible for the safety of lives, property, or the environment, and so software correctness must be determined with a high degree of certainty. To that end, simply testing a CPS is insufficient, as its interactions with the physical world may be difficult to predict, and unsafe conditions may not be immediately obvious. Formal verification can provide stronger safety guarantees but relies on the accuracy of the verified system in representing the real system. Bringing together verification and implementation can be challenging, as languages that are typically used to implement CPS are not easy to formally verify, and languages that lend themselves well to verification often abstract away low-level implementation details. Translation between verification and implementation languages is possible, but requires additional assurances in the translation process and increases software complexity; having both in a single language is desirable. This paper presents a formalization of MARVeLus, a CPS language which combines verification and implementation. We develop a metatheory for its synchronous refinement type system and demonstrate verified synchronous programs executing on real systems.

CCS Concepts: • **Software and its engineering → Formal software verification**; *Data flow languages*; • **Computer systems organization → Robotics**; **Embedded systems**; • **Theory of computation → Type structures**; **Operational semantics**.

Additional Key Words and Phrases: synchronous programming, refinement types, robotics

ACM Reference Format:
Jiawei Chen, José Luiz Vargas de Mendonça, Bereket Shimels Ayele, Bereket Ngussie Bekele, Shayan Jalili, Pranjal Sharma, Nicholas Wohlfeil, Yicheng Zhang, and Jean-Baptiste Jeannin. 2024. Synchronous Programming with Refinement Types. *Proc. ACM Program. Lang.* 8, ICFP, Article 268 (August 2024), 35 pages. https://doi.org/10.1145/3674657

*Work performed while at the University of Michigan

Authors' Contact Information: Jiawei Chen, University of Michigan, Ann Arbor, USA, chenjw@umich.edu; José Luiz Vargas de Mendonça, University of Michigan, Ann Arbor, USA, joselvdm@umich.edu; Bereket Shimels Ayele, Addis Ababa Institute of Technology, Addis Ababa, Ethiopia, bereket-shimels.ayele1@louisiana.edu; Bereket Ngussie Bekele, Addis Ababa Institute of Technology, Addis Ababa, Ethiopia, bereket.bekele@mail.utoronto.ca; Shayan Jalili, University of Michigan, Ann Arbor, USA, sjalili@umich.edu; Pranjal Sharma, University of Michigan, Ann Arbor, USA, spranjal@umich.edu; Nicholas Wohlfeil, University of Michigan, Ann Arbor, USA, njwohlf@umich.edu; Yicheng Zhang, University of Michigan, Ann Arbor, USA, zyicheng@umich.edu; Jean-Baptiste Jeannin, University of Michigan, Ann Arbor, USA, jeannin@umich.edu.

ACM 2475-1421/2024/8-ART268
https://doi.org/10.1145/3674657

1 Introduction

Much of the software we encounter, for example in automobiles, airplanes, or appliances, may interact with its physical environment, and thus are cyber-physical systems (CPS). Many CPS, such as those controlling vehicles or industrial processes, may pose great risk of harm in the event of a software failure. The development of safe CPS is therefore essential but is complicated by the difficulty of testing such systems, since their physical nature makes it difficult to exhaustively test all possible system and environmental configurations.

To this end, formal verification is an attractive approach that can be used to ascertain system safety. Hybrid automata [1] are commonly used to model CPS, though they do not directly represent executable programs. Other work in verifying CPS include Kind [31] and Kind 2 [17], which generate invariants for discrete-time synchronous programs using an incremental, bounded SMT-based approach. Although formal verification is more thorough than testing, it carries its own set of challenges. Mainstream programming languages common in CPS and embedded software development are challenging to verify, as their semantics are often not well-defined or precise enough. On the other hand, languages primarily designed for formal verification might abstract away too many implementation-level details of the original system to generate executable programs [1, 43]. Translation between verification and execution languages is possible but may introduce bugs of its own if unable to capture subtleties of the original system [37]. Verified translation is possible [8], though translation of any kind adds complexity to the development and debugging workflow. Combining verification and execution into a single programming language avoids some of the compromises of either approach, giving CPS developers the assurance that the properties they verify carry over to the real system.

The idea of languages that permit both verification and execution is not new; indeed, projects like Koord [29], VeriDrone [37] and VeriPhy [8] offer verification and execution of CPS, with special consideration given towards ensuring run-time behavior reflects what was verified. In Koord, multi-agent robotics systems are modeled in a language with formal semantics that allows direct verification and simulation. Programs can execute via a middleware layer [30] that replicates shared variables and interfaces with platform-specific controllers on real robots. System evolution follows a round-based approach that alternates between system and environment evolution, solving the common issue of synchronizing logical with physical time in a hybrid system model [29]. VeriPhy [8] describes a method for constructing CPS models and specifications in differential dynamic logic (dL) that can be soundly translated into a runtime monitor . This allows the safe mixing of a more optimal, but perhaps occasionally unsafe controller, with a safer alternative, though the method assumes such a safe controller exists. In comparison the present work focuses on compile-time verification of safety properties directly within an executable CPS language.

One approach to compile-time verification is to encode desired properties in the language's type system. As a result of the Curry-Howard-Lambek isomorphism relating data types to propositions, a derivation of said augmented type directly implies the provability of the property. Dependent- and refinement-type systems such as that of F* [51] or Liquid Haskell [34], enable type constraints to depend on program terms, and thus allow one to statically reason about a program's runtime behavior. A series of syntax-directed type-checking rules can then be used to generate constraints for a given program and its specification, the validity of which implies satisfaction of the specification.

This paper presents a formalization of the MARVeLus (**M**ethod for **A**utomated **R**efinement-type **Ve**rification of **Lus**tre) type system. In existing work [20], typing rules for MARVeLus were proposed, along with an implementation of a verified robot collision avoidance controller with stationary obstacles . However, a proof of type safety had yet to be developed for this language. In this paper, we present the following enhancements to MARVeLus over existing work [20]:

(1) A formalization of MARVeLus, introducing more precise operational semantics based on comparable synchronous languages, the introduction of a subset of Linear Temporal Logic (LTL) trace predicates for type specifications[15, 16, 21, 41, 47], and adjustments to the existing type system to enable a proof of type preservation;
(2) A definition and proof of type safety for the synchronous refinement type metatheory;
(3) Improved experimental methods supporting dynamic obstacles, such as collision avoidance when following a moving vehicle.

We are primarily focused on the correct formalization of a type-safe executable synchronous programming language. Although existing works present proof automation [10] or invariant generation techniques [17] for similar languages, we note that these developments are orthogonal to our present goals.

This paper is organized into five main parts. Section 2 introduces the MARVeLus language and presents a motivating example. Section 3 covers preliminary concepts and methods employed by MARVeLus for verification. Sections 4, 5, and 6 respectively describe the syntax, semantics, and type system that comprise our improved formalism for MARVeLus, including a definition for type preservation in synchronous programs. Sections 7 and 8 detail the implementation of MARVeLus atop an existing synchronous language. Finally, Sections 9 and 10 present the methods employed to execute MARVeLus on hardware, including an experiment with physical robots. We then conclude with a discussion of related work and future extensions.

2 Overview

MARVeLus enables CPS developers to verify and deploy programs to real-world CPS. By leveraging the existing synchronous program simulation and compilation infrastructure of Zélus [7], the language benefits from the great care already taken in ensuring its accuracy in modeling synchronous programs [6, 12]. To this, we add automated verification of program properties expressed as type annotations inspired by refinement types [48, 53].

2.1 A Simple Industrial Controller

We present a simplified verification and implementation workflow to illustrate how one might use the MARVeLus extensions in designing CPS software. Suppose one wants to design an industrial controller responsible for maintaining the liquid level in a tank, similar to the scenario described by Kamburjan [35]. The tank has a maximum capacity of 20 L and a fixed outflow rate of 0.1 L/s. We require that the tank level stays within 1 L and 19 L to prevent the tank from completely emptying or overflowing. The controller switches a solenoid valve, which can only either be completely open or closed, and can deliver 0.5 L/s when open. We design the controller so that it opens the valve when the tank level drops below 1.5 L and closes the valve when it exceeds 18.5 L, to afford a 0.5 L safety margin. For the sake of simulation, we assume this is a discrete-time system, with a constant time step of 0.1 s.

A verified MARVeLus implementation is shown in Fig. 1. Lines 1-5 define system constants such as the time step, maximum flow rate, and the tank's outflow rate. We refine these variables with type annotations, such as assuming they are positive, and that the maximum tank level must be greater than the minimum level. Lines 11-14 describe the state variables and their evolutions as a stream of tuples, of which each element represents the flow rate and liquid level at each instant. The type annotation on the tuple requires that both elements are floats, and that together they satisfy the conditions given in Lines 8-10. Note that we use `f` and `l` to refer to the first and second elements of the tuple to distinguish them from their instantiations in the remainder of the program.

```
1  let dt:{v:float | v > 0.} = 0.1
2  let maxflow:{v:float | v > 0.} = 0.5
3  let outflow:{v:float | v > 0.} = 0.1
4  let minlevel:{v:float | v > 0} = 1.
5  let maxlevel:{v:float | v > minlevel} = 19.
6  let node exec ()  : {(vf:float)*(vl:float) | true} =
7    let rec ((flow, level):{(f:float)*(l:float) |
8        ((l >= minlevel && l <= maxlevel) &&
9        ((l <= (maxlevel -. dt *. (maxflow -. outflow)) && f = maxflow)
10         || (l >= (minlevel +. dt *. outflow) && f = 0.)))}) =
11     (0., 15.) fby
12     ((if (level <= (minlevel +. 0.5)) then maxflow else
13       (if (level >= (maxlevel -. 0.5)) then 0. else flow)),
14       (level +. (flow -. outflow) *. dt) models (robot_get ("level")))
15     in (() models robot_str ("flow", flow)); (flow, level)
```

Fig. 1. The industrial controller code with added refinements and external interfaces

The safety condition is specified on Line 8, requiring that the tank level always remain between `minlevel` and `maxlevel`. However, this specification alone is not sufficiently strong to verify the system, as it is not inductive. That is to say, proving the satisfaction of the safety specification now does not allow one to prove safety for the next time interval. Provided only this specification, the verifier produces counterexamples, even if they are not physically feasible. For instance, if the liquid level is at exactly 19 L and the valve was open in the previous instant, the controller cannot respond quickly enough to close the valve before the tank overflows. However, we know that the system has started in a safe state, and that the controller will not have allowed the liquid level to reach 19 L with an open valve. Therefore, we must specify an additional invariant (Lines 9-10) to assume that the controller will have made the correct decision in the previous instant. Specifically, we assume that the controller in the previous instant has left us with at least one time step to react and prevent an unsafe condition. By inspecting the controller code, one can assure themselves that this assumption is indeed satisfied. The verifier can then use this fact to check that the controller will continue to make the correct decision in the next instant, thus proving the property inductively.

Line 11 sets the initial condition of the system; we assume the tank level begins at 15 L and the valve is initially closed. Lines 12-13 describe the controller's behavior and its effect on the commanded flow rate. The system reacts if the level approaches the limits and otherwise retains the previous flow rate. Line 14 then evolves the physical system, simulating the tank's liquid level based on inflow and outflow rates, or retrieving the actual tank level from a sensor labeled "level". We use the `models` syntax to separate the simulation model (on the left) and the actual implementation (on the right); only the left side is verified, and we assume that the supplied model accurately reflects the real system dynamics. Line 15 then publishes the commanded flow rate to an actuator labeled "flow", and returns the flow rate and level to be used in other parts of the program.

3 Preliminaries

3.1 Synchronous Programming

Synchronous programming treats all program data as streams which may take on different values at any given moment. Program computations are treated as manipulations to these streams to influence their evolution in time. Streams are well-suited for modeling embedded systems, which

often handle streams of inputs and outputs in real time [27]. For example, an autonomous vehicle must constantly interpret sensor data and produce control commands in a timely fashion to avoid an accident. Synchronous programming has gained significant traction in industry, where tools such as SCADE (a derivative of Lustre) [22] and Esterel [13] have been entrusted with designing integrated circuits, avionics, and other critical systems. Thus, a verified synchronous language has the advantage of familiarity for eventual industrial adoption.

Synchronous programming observes logical time, in which a clock cycle is determined by certain actions or events involving the system, which may be decoupled from physical ("wall-clock") time. All computations in a synchronous program must finish before the next clock cycle; thus, they are synchronized by this basic unit of time. This lends itself well to the design of embedded systems, as the data-flow model of synchronous programming closely resembles the flow of signals in electronic circuits [32] and imposes time constraints on computations. A complication that arises from combining software with the physical environment is the reconciliation of logical and physical clocks. In this paper, we focus only on discrete systems and reserve the examination of verifying systems spanning these two time domains for future work, noting that significant work has already gone into accurately simulating hybrid programs in Zélus [7]. We discretize the dynamics and clock our synchronous program at the rate of discretization. Since MARVeLus is primarily concerned with the types of streams and the values they emit, we assume that all streams have compatible clocks, a property already statically verified by the Zélus compiler.

Another useful property of synchronous programs is that they operate in bounded memory [12, 14], which limits the number of previous state values that are allowed to be observed at any time. This precludes, for instance, the arbitrary storage of sensor values, and makes memory usage more predictable. This is especially important for embedded systems, which often have limited computational resources. By construction, streams can only access a fixed number of previous states to derive the next state.

Lustre, and by extension Zélus, also observe a notion of causality, meaning that a program may only depend on values available in past cycles, and on present values in a way that does not produce circular dependencies. For instance, a stream may not instantaneously depend on its own value in the current cycle, but may depend on the value of another stream, provided the other stream does not already depend on it. Causality is a useful property for a CPS modeling language as it ensures systems created in the language are physically feasible. Causality is checked statically by the Zélus compiler and has already been formalized as a dedicated causality-analysis type system in existing work [6, 23], so we henceforth assume that programs we check already obey causality with respect to this formalism. To this end, we assume that a term e is always well-typed in the causality type system [23].

3.2 Refinement Types

Refinement types enhance the standard type system by allowing types to be specified using logical predicates that can take values of terms as inputs. These predicates serve as constraints on the base type such that all terms that inhabit the refined type satisfy the predicate. For instance, positive floating-point numbers can be specified using the refinement type $\{w : \text{float} \mid w > 0.\}$

Refinement types belong to the broader field of dependent types, which are, generally speaking, types that can depend on the values of terms [42]. Although full dependent type theory has the advantage of expressiveness [51], refinement types lend themselves well to generating verification conditions that depend on terms, while remaining decidable if restricted to the theories of equality, uninterpreted functions, and real arithmetic [48]. As a result, refinement types lose the expressivity of allowing arbitrary functions in refinements, abstracting away the bodies of non-arithmetic functions [48], which if interpreted would jeopardize decidability [34].

v	$::= c$	Literal Values
	$\mid (v_1,\ v_2 ...,\ v_n)$	Tuples
e	$::= v$	Values
	$\mid \mathsf{let}_h\ (x : \tau) = e_1\ \mathsf{in}\ e_2$	Local Bindings
	$\mid \mathsf{let\ rec}_h\ (x : \tau) = e_1\ \mathsf{in}\ e_2$	Recursive Bindings
	$\mid f\ x$	Application
	$\mid e_1\ \mathsf{fby}\ e_2$	Stream Composition
	$\mid \mathsf{delay}(e)$	Delayed Evaluation
	$\mid (e_1,\ e_2 ...,\ e_n)$	Tuples of Expressions
	$\mid \mathsf{if}\ x_c\ \mathsf{then}\ e_t\ \mathsf{else}\ e_f$	Conditionals
	$\mid e\ \mathsf{models}\ r$	Modeling
r	$::= \mathsf{robot\_get}\ k$	Robot Get
	$\mid \mathsf{robot\_str}\ k\ e$	Robot Store
d	$::= \mathsf{let}\ f\ (x : \tau_1) : \tau_2 = e$	Named Function Definition

Fig. 2. MARVeLus Syntax

3.3 Verification Via Typing

Verification via typing relies on the Curry-Howard-Lambek isomorphism to relate desired program properties (propositions) with types, and thus property checking becomes type checking [42]. Furthermore, type-checking is performed statically at compile-time, so verification at this stage in compilation has the chance to catch bugs in the program before any code is actually executed. This is particularly attractive to CPS, where testing on a real system can be difficult, risky, or costly. Verification via typing also allows our verifier implementation to be modular; all terms still possess a base type that can be checked using the existing type checker, while terms augmented with type refinements can provide additional information or conditions to the verifier. This allows the base Zélus type checker to remain independent of our additions. Refinement types [34] provide a method of adding annotations to the type system while maintaining decidability. An additional benefit of introducing verification through type-checking is that it promotes modularity in user code; terms that have been type-checked can then be re-used in other parts of the source code, carrying with them the proof of safety [25]. This can be especially important with large and/or long-term projects requiring extensive integration or regression testing of patches or updates.

4 Syntax

The stream syntax (Fig. 2) is modeled around Lustre streams, which forms a discrete-time subset of the existing Zélus syntax [7]. Similarly to other synchronous languages [15], we make the distinction between values v, which are constants emitted by stream expressions. We introduce a new entry h ("history") in recursive and non-recursive local bindings, which embeds a bound variable's past values into the syntax. This is essential for our operational semantics as stream

expressions can depend on values that have occurred in the past and thus we may need to recover those values to re-construct the appropriate derivation tree. A syntactic sugar we employ is allowing local bindings to be defined without histories; these are considered uninitialized and are implicitly assigned empty histories ($h = []$) Similarly to other refinement type systems [34, 48], we require that certain constructs take variable arguments (such as function application and conditionals) to simplify the metatheory by preventing the introduction of arbitrary expressions into refinement predicates [34]. Another syntactic sugar we employ is to allow certain non-variable terms syntactically considered "safe" for the refinement logic to be used as argument functions (such as simple arithmetic expressions and conditionals), which greatly simplifies programs written in the real-world implementation of the language. We note that Zélus only supports named function definition at the top-level [7], which we replicate.

Some elements of the syntax are not user-facing and are exclusively to support internal functionality. The delay() operator is one such case, which allows an expression's evaluation to be deferred for one instant while preserving its original behavior despite environment updates. Likewise, we only allow let-bindings (both recursive and non-recursive) without initialized histories (i.e. $h = []$) in the user-facing portion of the language. This ensures prevents arbitrary values from being introduced to histories.

As part of the robotics extensions to Zélus, we introduce built-in keywords to facilitate reading and writing values to hardware (r) as shared variables labeled by string constants k. Additionally, we introduce the syntax e models r which allows one to define two implementations for a stream—a deterministic model for simulation and verification, and the implementation involving real hardware. As it is impossible to predict or verify at compile-time the exact outputs produced by real sensors, the CPS designer is responsible for ensuring that the model accurately represents actual behavior . In addition to the robotics extensions, we introduce the syntax necessary to support refinement types in Zélus.

4.1 Specification Syntax

The core of the specification syntax (Fig. 3) is derived from similar refinement type systems [34, 48]. Typically, refinement type systems require that refinement predicates come from decidable logics such as the quantifier-free logic of equality, uninterpreted functions, and linear arithmetic. We note that allowing nonlinear polynomial real arithmetic maintains decidability, as shown by Tarski [52]. We find this addition useful for specifying the kinematics of physical systems and it is supported by the Z3 solver [24].

A type can either be a base type b, which represents a stream consisting solely of values belonging to a particular primitive data type, or refinement types. Refinement types of the form $\{w : b \mid \varphi\}$ allow one to specify streams that satisfy a predicate φ, and introduce a value variable w which binds to any term inhabiting the type, allowing φ to reference terms [48]. For example, one may write the type of non-negative integers as $\{w : \mathsf{int} \mid \mathsf{w} \geq 0\}$. As a natural extension of standard refinement types, predicates in our refinement type specifications become streams of Boolean values.

One of our main contributions over existing work [20] is extending the predicate syntax of MARVeLus to explicitly accommodate select temporal predicates from Linear Temporal Logic (LTL) [45]. The refinement predicates discussed thus far form the syntax of "state predicates", which in the absence of temporal operators, reason about the immediate value of a stream at the current instant. To this, we add the temporal operators "Always" ($\Box$) and "Next" ($\bigcirc$), along with the conjunction of predicates containing these operators. This expanded set comprises the full set of specifications we allow as refinement predicates in the type syntax. While LTL normally includes other temporal operators along with disjunction at the top-level, we choose to focus primarily on safety properties which can be expressed globally (i.e. with $\Box$), which allows proofs to be inductive. In practice this

$$
\begin{array}{rll}
\tau & ::= b \mid \{v : b \mid \varphi\} \mid \tau \rightarrow \tau & \text{Types} \\
b & ::= \mathsf{int} \mid \mathsf{float} \mid \mathsf{bool} \mid b_1 \times b_2 \times \ldots \times b_n & \text{Base Types} \\
p, q & ::= \mathsf{true} \mid \mathsf{false} \mid x \mid e_1 = e_2 \mid e_1 > e_2 \mid p \wedge q \mid \neg p & \text{State Predicates} \\
\varphi, \psi & ::= p \mid \Box\varphi \mid \bigcirc \varphi \mid \varphi \wedge \psi & \text{Trace Predicates}
\end{array}
$$

Fig. 3. Refinement Type Syntax, where we note that e_1, e_2 are terms within the logic of quantifier-free equality, uninterpreted functions, and arithmetic (QF-EUFA)

$$
\begin{array}{rcl}
\sigma & ::= & \varnothing \mid \sigma, x = h \\
S & ::= & \varnothing \mid S, f(x) = e \\
h & ::= & [] \mid h :: v \\
\mathsf{prev}(h :: v) & \triangleq & h \\
\mathsf{prev}(\sigma) & \triangleq & \{\mathsf{prev}(\sigma(x)) \mid x \in \mathsf{dom}(\sigma)\}
\end{array}
$$

Fig. 4. Definitions for function environment S, term environment σ, and the histories h of variables bound in σ

is often sufficiently rich for describing most desirable behaviors of safety-critical CPS, as these properties typically require specifying that something either always or never happens.

5 Stream Semantics

MARVeLus programs operate on streams of data, producing values at each instant. Although the streams themselves do not terminate, they may emit a value at a given instant representing the stream's current state. Lustre's clock calculus allows streams that do not emit values on all clock cycles [32], but we currently consider only streams that are compatible with the basic clock, i.e. streams that always emit a value on every cycle of the most frequent clock.

We describe the operational semantics of MARVeLus with the judgement $S;\ \sigma \vdash e \overset{v}{\hookrightarrow} e'$, which represents the stream e in stream context σ and function context S that simultaneously transitions into the new stream e' and emits a value v at each clock cycle. Similarly to other interpretations of Lustre semantics [47], the context σ is comprised of mappings between variables and histories of their past values (Fig. 4). That is, for any $x \in \mathsf{dom}(\sigma)$, $\sigma(x) = h$ where $h = v^{-n} :: v^{-n+1} \ldots v^0$ is a history of the past values of x, up to the immediately preceding time instant.

Consistent with the design of Lustre [32], loops and recursive function calls are excluded. As a result, the evaluation of a stream e at any instant is terminating, meaning a value v is always emitted on a transition of e.

One of the challenges in deriving operational semantics arose from reconciling the behavior of recursively-defined streams (let rec) with stream composition (fby). In an earlier iteration of the semantics [20], the left and right hand sides of (fby) are evaluated simultaneously. When a stream depends on its past values, it necessarily employs fby to both initialize the stream and delay self-references so that they do not result in unbounded recursion. For example, in the program let rec $x = 0$ fby $x + 1$ in x, x is defined initially as 0, and then for all subsequent instants is defined as the sum of 1 and the past value of x. This is not straightforward to characterize in a syntax-guided manner. Part of the reason is that, when evaluating 0 fby $x + 1$, the notion of the left-hand side

becoming the initial value of x is lost. This information is only known at the level of the recursive let-binding. This is problematic if the left and right sides of 0 fby $x + 1$ must be evaluated together, because $x + 1$ cannot be computed unless x is known to be initially 0, which is not known until 0 fby $x+1$ is fully evaluated! In other (non-operational) semantics [15, 16], this was in part resolved by deferring the computation of the right-hand side of fby (or its semantic equivalent). However, simply deferring the computation reveals a new problem: if the deferred expression contains free variables that have updated since the delay occurred, the behavior of this expression will change so that it no longer behaves like a properly delayed version of itself. Prior semantics address this by either storing intermediate values for delayed terms (akin to a buffer) [16], or by leaving variables symbolic [15]. Our operational approach takes inspiration from the former, in that local bindings maintain a history of their variables' past values so that deferred computations can refer back to the states of variables from the time before the delay occurred. Finally, one more issue to address is preserving this history in a syntactical manner. At each time instant, the operational semantics for the next step is derived independently as a new derivation tree. As such, the derivation needs to be re-populated with the past values of variables from the previous derivation so that the histories of variables can be recovered. This motivated the changes we made to the semantics of fby and let-bindings, and is essential for a syntax-directed operational semantics.

5.1 Summary of Stream Behaviors

Drawing inspiration from Lustre and Esterel [21, 41, 47], we define the operational semantics of MARVeLus in Fig. 6

(1) *Constant Streams*: These are literals lifted from the base language; they return a fixed value each instant. Lustre treats all primitives as constant streams (i.e. 5 in Lustre is the constant stream that always emits 5) [14], and we adopt this convention as well.
(2) *Variables*: Variables return the most recent value bound to them in σ. A variable x steps to x because variables themselves do not update; instead their entry in σ is updated each instant to reflect the value of the stream to which it was originally bound; this update is performed by (S-LET) and (S-LETREC), the two places where new variables are bound.
(3) *Pointwise Applications of Functions to Argument Streams*: Similarly to how a function can be mapped over an array, we define the pointwise application of a named function f on an argument variable x as $f\ x$, the resultant stream being f applied to the most recent value bound to x. Functions are imported from the non-stream base language and are assumed to be well-typed with respect to that language. The semantics are defined by the rule (S-APP).
(4) *Compositions of Streams*: Lustre and Zélus allow a stream to be "attached" to another via s fby t (pronounced "followed-by"), resulting in a new stream consisting of the initial value of s followed by the stream t delayed by one clock, i.e. $s_0,\ t_1,\ t_2, \ldots$. The behavior of fby is defined by the rule (S-FBY). When the expression e_1 fby e_2 is evaluated, it emits the value that e_1 emits when it is evaluated, and then rewrites to $\mathsf{delay}(e_2)$. While intuitively one may think the result should simply be e_2, as the purpose of fby is to defer computation of the second computation, it is possible that free variables occur in e_2. This requires special treatment because the values of these free variables may have updated during the previous evaluation, and so e_2 evaluated in the current instant may not produce the same values it did in the previous instant. The intent of fby is to "delay" the second expression by one instant, so that it produces the same values it would have had it not been delayed, but one instant later. Essentially this "shifts" its behavior in time while also accounting for variables that may be assigned new values in the meantime. This operation could only be done through a special

delay() operator and corresponding (S-DELAY) semantics, which bridges the past and the present to allow expressions and their behaviors to be preserved in a newer context.

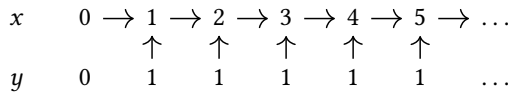
(5) *Delaying Streams*: Streams that are delayed by fby may contain variables which were bound outside the scope of the fby. Consider the program in Fig. 5*a*. Note how the variable y is used inside the scope of a fby (in $x + y$), but y is bound outside the scope of this delay. This means that y is continually assigned new values by the evaluation of 0 fby 1, which is not under a delay. If fby simply deferred evaluation of $x + y$ without considering that variables will have updated in the meantime, the first evaluation of $x + y$ will have skipped the initial value of y, putting x and y out of sync with one another. Furthermore, x will instantaneously depend on itself. Therefore, evaluation of fby is accompanied by delay, which re-evaluates expressions in the previous context. Because previous values of variables in σ are retained in let and let rec, the only places that bind new variables, it is possible to "rewind" σ to previous states simply by removing the last value from each entry. In other words, the previous σ, $\mathsf{prev}(\sigma)$ is always completely recoverable from σ. This is encapsulated by prev() operator (Fig. 4). Variables that are delayed are guaranteed to have a value available in its corresponding context, since they are updated at least as many times as delay is applied through (S-FBY), since each delay will have been accompanied by an evaluation of the outer sub-expression which bound the variable. Therefore, it is impossible to revert σ to a state before the "beginning of time" for a program.

(6) *Robot-Specific Expressions*: The "robot expressions" robot_get, robot_str, and models serve to connect the rest of the semantics with real-world hardware and are largely semantically transparent, with the exception of combining robot_get with models when a program is running in "robot-mode". In this mode, the program takes in real sensor values using robot_get. Since it is impossible to predict the values supplied by real-world sensors and because robot_get can only be present on the right-hand side of models, robot_get as well as models expressions in robot-mode have no operational semantics. A models expression while not in robot-mode (as indicated by the side condition) is completely transparent to the underlying stream expression, and thus takes on identical semantics. robot_str is assumed to always be a successful write operation performed on a piece of robot hardware, but has no other effect on the MARVeLus program.

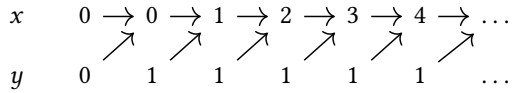
(7) *Stream Recursion*: Streams can be defined recursively in terms of themselves. A stream $\mathsf{let\ rec}\ x = e_1\ \mathsf{in}\ e_2$ allows x to occur in e_1 under special circumstances. Although recursively defined streams syntactically resemble recursive definitions in conventional languages, their semantics are entirely different. As with Lustre, MARVeLus considers variables to be identically equal to their definitions [32]. Consequently, one must be mindful of circular dependencies in definitions. Streams must not be defined in terms of their current values, as one may otherwise produce contradictions ($\mathsf{let\ rec}\ x = x + 1\ \mathsf{in}\ x$) or nondeterministic behavior ($\mathsf{let\ rec}\ x = x\ \mathsf{in}\ x$). Both scenarios have "instantaneous dependencies" since the value of x depends on its current value. These are not well-formed programs and are statically rejected. Instead, (S-LETREC) allows the evaluation of e_1, whose value will determine the current value of x, to reference x up to and including its previous value, but not its current value. A nil is inserted at the end of $\sigma(x)$ since the last position is reserved for the current value of a variable (which depends on the current evaluation). The evaluation of e_2 is then allowed to reference the current value of x, as one would expect in a local binding. Recursively defined streams must delay references to themselves using the fby keyword, which also defines the stream's initial value(s). This ensures that streams are causal, depending on strictly prior values. The stream $\mathsf{let\ rec}\ x = 0\ \mathsf{fby}\ x + 1\ \mathsf{in}\ x$ is well-formed, because the recursive reference to x in its definition occurs behind a delay. This stream initially produces the value 0, and then for

```
let rec x =
  (let y = 0 fby 1 in (0 fby x+y))
  in x
```

(a) A program that contains variables used inside the scope of fby



(b) Improperly delayed variables. The arrows represent the values of x and y that are used when computing the next value of x. Note how the first value of y is skipped in computing $x + y$



(c) Properly delayed variables. No values of y are skipped when computing $x + y$.

Fig. 5. Traces of the variables x and y in a stream program (5a), if the variables are improperly (5b) and properly (5c) delayed

all subsequent evaluations produces a value one more than its previous value. The variable x is annotated with a type τ to simplify type checking and this type is likewise "stepped" to reflect the behavior of x in subsequent instants. The operator tl() and the semantics of "stepping" types is described in Section 6.

(8) *Local Bindings*: This expression type is similar to that of stream recursion, except the evaluation of e_1 must not depend on x, as this is a non-recursive definition. Otherwise, all other behaviors, including stepping the type annotation of x, are the same.

(9) *Branching*: Branching in MARVeLus resembles a multiplexer in a digital circuit, in that the conditional selects the value of one of the two streams to emit. That is, the stream if x_c then e_t else e_f takes the current value of the "true" branch e_t or the "false" branch e_f depending on the truth value of x_c. As before, we assume all three streams have compatible clocks. The two rules (S-IF-T) and (S-IF-F) correspond respectively to the cases when x_c emits true or false, and thus whether the stream emits the value of e_t (thus emitting v_t) or e_f (thus emitting v_f). The conditional does not affect the behavior of the branches themselves; the branch not taken will transition as well as reflected in the rules. This prevents the stream that is not selected from delaying indefinitely.

5.2 Predicate Semantics

MARVeLus allows one to define predicates in the style of Linear Temporal Logic [45]. Adopting the notation of Manna and Pnueli [38], we use $\vDash$ to denote satisfaction of a property over a stream and $\Vdash$ to denote satisfaction over a single state. In this section, we denote streams s as sequences of values s_i for illustrative purposes. Furthermore, we note that predicates themselves are in fact streams of Boolean values in our system, with each value representing the predicate's satisfaction given the state of the stream at that instant. Since specifications in MARVeLus are made with respect to the initial definitions of streams (i.e. before program execution), predicates written by the user are considered over the entire time period of the program, from the beginning of program execution onwards.

$\boxed{S;\ \sigma \vdash e \overset{v}{\hookrightarrow} e'}$

$$\frac{}{S;\ \sigma \vdash c \overset{c}{\hookrightarrow} c}\ \text{(S-CONST)} \qquad \frac{}{S;\ \sigma, x = h :: v \vdash x \overset{v}{\hookrightarrow} x}\ \text{(S-VAR)}$$

$$\frac{S;\ \sigma \vdash e_1 \overset{v_1}{\hookrightarrow} e_1'}{S;\ \sigma \vdash e_1 \text{ fby } e_2 \overset{v_1}{\hookrightarrow} \text{delay}(e_2)}\ \text{(S-FBY)} \qquad \frac{S;\ \text{prev}(\sigma) \vdash e \overset{v}{\hookrightarrow} e'}{S;\ \sigma \vdash \text{delay}(e) \overset{v}{\hookrightarrow} \text{delay}(e')}\ \text{(S-DELAY)}$$

$$\frac{S;\ \sigma, x = h :: \text{true} \vdash e_t \overset{v_t}{\hookrightarrow} e_t' \qquad S;\ \sigma, x = h :: \text{true} \vdash e_f \overset{v_f}{\hookrightarrow} e_f'}{S;\ \sigma, x = h :: \text{true} \vdash (\text{if } x \text{ then } e_t \text{ else } e_f) \overset{v_t}{\hookrightarrow} (\text{if } x \text{ then } e_t' \text{ else } e_f')}\ \text{(S-IF-T)}$$

$$\frac{S;\ \sigma, x = h :: \text{false} \vdash e_t \overset{v_t}{\hookrightarrow} e_t' \qquad S;\ \sigma, x = h :: \text{false} \vdash e_f \overset{v_f}{\hookrightarrow} e_f'}{S;\ \sigma, x = h :: \text{false} \vdash (\text{if } x \text{ then } e_t \text{ else } e_f) \overset{v_f}{\hookrightarrow} (\text{if } x \text{ then } e_t' \text{ else } e_f')}\ \text{(S-IF-F)}$$

$$\frac{S;\ \sigma, x = h :: \text{nil} \vdash e_1 \overset{v}{\hookrightarrow} e_1' \qquad S;\ \sigma, x = h :: v \vdash e_2 \overset{w}{\hookrightarrow} e_2'}{S;\ \sigma \vdash \text{let rec}_h\ x : \tau = e_1 \text{ in } e_2 \overset{w}{\hookrightarrow} \text{let rec}_{h::v}\ x : \text{tl}(\tau) = e_1' \text{ in } e_2'}\ \text{(S-LETREC)}$$

$$\frac{S;\ \sigma \vdash e_1 \overset{v}{\hookrightarrow} e_1' \qquad S;\ \sigma, x = h :: v \vdash e_2 \overset{w}{\hookrightarrow} e_2'}{S;\ \sigma \vdash \text{let}_h\ x : \tau = e_1 \text{ in } e_2 \overset{w}{\hookrightarrow} \text{let}_{h::v}\ x : \text{tl}(\tau) = e_1' \text{ in } e_2'}\ \text{(S-LET)}$$

$$\frac{}{S, f(x) = e;\ \sigma, y = h :: v \vdash f(y) \overset{e[x \mapsto v]}{\hookrightarrow} f(y)}\ \text{(S-APP)}$$

$$\frac{S;\ \sigma \vdash e \overset{v}{\hookrightarrow} e' \qquad \neg\ \text{robot-mode}}{S;\ \sigma \vdash e \text{ models } r \overset{v}{\hookrightarrow} e' \text{ models } r}\ \text{(S-MODELS)}$$

Fig. 6. MARVeLus Stream Semantics, inspired from existing semantics of Lustre and Esterel [16, 21, 41, 47]

5.2.1 State Predicates. For a state predicate p and stream s, $(s_i,\ s_{i+1},\ s_{i+2}, \ldots) \vDash p$ if and only if $s_i \Vdash p$.

5.2.2 Trace Predicates. Trace predicates offer a more holistic view of the stream's behavior, and we define a subset of those found in Linear Temporal Logic.

(1) Globally ($\Box$): A property $\Box\varphi$ requires that φ be satisfied for all values of a stream beginning at the present instant. That is, $(s_i,\ s_{i+1},\ s_{i+2}, \ldots) \vDash (\Box\varphi)_i \iff \forall (j \geq i)\ .\ s_j \Vdash \varphi_i$. Verification of these predicates in MARVeLus is handled via induction; the left- and right-hand sides of a fby construct become the initial condition and inductive step, respectively. Since refinement predicates are checked in the program's initial instant, satisfaction of $\Box\varphi$ implies satisfaction of φ for all instants in time; thus $\Box$ properties are invariant properties.
(2) Next ($\bigcirc$): A property $\bigcirc\varphi$ requires that φ be satisfied in the next instant. In other words, $(s_i,\ s_{i+1},\ s_{i+2}, \ldots) \vDash \bigcirc\varphi \iff s_{i+1} \Vdash \varphi$

We additionally introduce hd() as a convenience function that transforms a trace predicate φ such that for any streams s and t, $s \vDash \varphi \Rightarrow (s \text{ fby } t) \vDash \text{hd}(\varphi)$. Thus, $\text{hd}(\varphi)$ represents a condition

$$\begin{aligned}
\mathsf{hd}(p) &\triangleq p & \mathsf{impl}(q, p) &\triangleq \neg q \vee p \\
\mathsf{hd}(\varphi \wedge \psi) &\triangleq \mathsf{hd}(\varphi) \wedge \mathsf{hd}(\psi) & \mathsf{impl}(q, \varphi \wedge \psi) &\triangleq \mathsf{impl}(q, \varphi) \wedge \mathsf{impl}(q, \psi) \\
\mathsf{hd}(\Box\varphi) &\triangleq \mathsf{hd}(\varphi) & \mathsf{impl}(q, \Box\varphi) &\triangleq \Box(\mathsf{impl}(q, \varphi)) \\
\mathsf{hd}(\bigcirc\varphi) &\triangleq \top & \mathsf{impl}(q, \bigcirc\varphi) &\triangleq \bigcirc(\mathsf{impl}(q, \varphi))
\end{aligned}$$

Fig. 7. Inductive definitions of the hd and impl operators. The impl operator is needed because the syntax only allows the $\vee$ operator (and thus implications) at the level of state formulas.

that a stream with s_0 as its initial value will satisfy, regardless of subsequent values, assuming that s satisfies φ. This is useful for deriving properties about a stream constructed using s fby t, if properties are known about s and t individually. The transformation is defined recursively for state predicates p and trace predicates φ, ψ in Fig. 7. The transformation for state predicates and $\Box$ are unsurprising; state predicates are only ever concerned with the immediate value, and $\Box\varphi$ follows from the equivalence $\Box\varphi \equiv \varphi \wedge \bigcirc\Box\varphi$. No additional information can be learned from $\bigcirc\varphi$ as its satisfaction tells us nothing about the immediate value of the stream. We also introduce the impl() operator to condition refinement predicates at each instant on a boolean stream, without introducing disjunctions of trace predicates, to maintain the ability to split predicates.

Lemma 1 (Correctness of hd). *For any stream $\sigma = (\sigma_0,\ \sigma_1, \ldots)$, if $\sigma \vDash \psi$, then for any other stream $\tau = (\tau_0,\ \tau_1, \ldots)$, $(\sigma_0,\ \tau_1,\ \tau_2 \ldots \tau_n \ldots) \vDash \mathsf{hd}(\psi)$*

Proof. By induction on the definition of $\mathsf{hd}(\psi)$
Case p where p is a state predicate. Assume $\sigma \vDash p$. Then $\sigma_0 \Vdash p$. Therefore, $(\sigma_0,\ \tau_1,\ \tau_2 \ldots \tau_n \ldots) \vDash \mathsf{hd}(p) \equiv p$ for any τ.
Case $\psi_1 \wedge \psi_2$. Assume $\sigma \vDash \psi_1 \wedge \psi_2$. Then $\sigma \vDash \psi_1$ and $\sigma \vDash \psi_2$. By IH, $(\sigma_0,\ \tau_1',\ \tau_2' \ldots \tau_n' \ldots) \vDash \mathsf{hd}(\psi_1)$ for any τ', and $(\sigma_0,\ \tau_1',\ \tau_2' \ldots \tau_n' \ldots) \vDash \mathsf{hd}(\psi_2)$ for any τ'. Therefore, $(\sigma_0,\ \tau_1',\ \tau_2' \ldots \tau_n' \ldots) \vDash \mathsf{hd}(\psi_1 \wedge \psi_2)$.
Case $\Box\psi$. Assume $\sigma \vDash \Box\psi$. By induction hypothesis (IH), $(\sigma_0,\ \tau_1',\ \tau_2' \ldots \tau_n' \ldots) \vDash \mathsf{hd}(\psi)$ for any τ'. Therefore, $(\sigma_0,\ \tau_1,\ \tau_2 \ldots \tau_n \ldots) \vDash \mathsf{hd}(\Box\psi) \equiv \mathsf{hd}(\psi)$ for any τ.
Case $\bigcirc\psi$. Trivial. $(\sigma_0,\ \tau_1,\ \tau_2 \ldots \tau_n \ldots) \vDash \mathsf{hd}(\bigcirc\psi) \equiv \top$ for any σ and τ. □

6 The MARVeLus Type System

Formal verification of MARVeLus programs relies upon type checking using specifications supplied as type refinements. Our primary contribution to the type system is the adaptation and implementation of refinement type specifications for streams. Unique to our refinement type system is the introduction of temporal operators in refinement predicates.

Fig. 8 gives the set of typing rules, adopting the proposed stream syntax.

6.1 Type Semantics

A stream's type refinement specifies the stream's behavior from the current instant onwards. For example, a stream inhabiting the type $\{w : \mathsf{int} \mid \Box(w \geq 0)\}$ is non-negative at the current instant, and will continue to be non-negative in all subsequent time instants. We follow the LTL convention in that predicates are checked for validity in the time instant in which they occur. Thus, a stream's type describes the stream's immediate value along with the values it will produce in the future. We use the same judgement as Zélus [7]–$G, H \vdash e : \tau$–to express that, in function context G and stream context H, an expression e has type τ. It has proven to be more manageable to separate imported functions from the base language into a distinct environment G.

$\boxed{G;\ H \vdash e : \tau}$

$$\frac{}{G;\ H \vdash c : \{w : b \mid \square(w = c)\}}\ \text{(T-CONST)} \qquad \frac{G;\ H(x) = \{w : b \mid \psi\}}{G;\ H \vdash x : \{w : b \mid \psi \wedge \square(w = x)\}}\ \text{(T-VAR)}$$

$$\frac{G;\ H \vdash e_1 : \{w : b \mid \mathsf{hd}(\psi_1)\} \qquad G;\ H \vdash e_2 : \{w : b \mid \psi_2\}}{G;\ H \vdash e_1\ \mathsf{fby}\ e_2 : \{w : b \mid \mathsf{hd}(\psi_1) \wedge \bigcirc\psi_2\}}\ \text{(T-FBY)}$$

$$\frac{G;\ \mathsf{prev}(H) \vdash e : \tau}{G;\ H \vdash \mathsf{delay}(e) : \tau}\ \text{(T-DELAY)}$$

$$\frac{G;\ H \vdash \tau_1 \qquad G;\ H \vdash e_1 : \tau_1 \qquad G;\ H, x : \tau_1 \vdash e_2 : \tau_2}{G;\ H \vdash \mathsf{let}_h\ x : \tau_1 = e_1\ \mathsf{in}\ e_2 : \tau_2}\ \text{(T-LET)}$$

$$\frac{G;\ H \vdash \tau_1 \qquad G;\ H, x : \tau_1 \vdash e_1 : \tau_1 \qquad G;\ H, x : \tau_1 \vdash e_2 : \tau_2}{G;\ H \vdash \mathsf{let}_h\ \mathsf{rec}\ x : \tau_1 = e_1\ \mathsf{in}\ e_2 : \tau_2}\ \text{(T-LETREC)}$$

$$\frac{G;\ H \vdash x_c : \mathsf{bool} \qquad \begin{array}{c} G;\ H \vdash e_t : \{w : b \mid \mathsf{impl}(x_c, \psi)\} \\ G;\ H \vdash e_f : \{w : b \mid \mathsf{impl}(\neg x_c, \psi)\} \end{array}}{G;\ H \vdash \mathsf{if}\ x_c\ \mathsf{then}\ e_t\ \mathsf{else}\ e_f : \{w : b \mid \psi\}}\ \text{(T-IF)}$$

$$\frac{G, f : (x : \{w_1 : b_1 \mid \varphi_1\} \to \{w_2 : b_2 \mid \varphi_2\});\ H \vdash y : \{w_1 : b_1 \mid \square\varphi_1\}}{G, f : (x : \{w_1 : b_1 \mid \varphi_1\} \to \{w_2 : b_2 \mid \varphi_2\});\ H \vdash f\ y : \{w_2 : b_2 \mid \square\varphi_2[x \mapsto y]\}}\ \text{(T-APP)}$$

$$\frac{G;\ H \vdash e : \tau}{G;\ H \vdash e\ \mathsf{models}\ r : \tau}\ \text{(T-MODELS)} \qquad \frac{}{G;\ H \vdash \mathsf{nil} : \tau}\ \text{(T-NIL)}$$

$$\frac{G;\ H \vdash e : \tau \qquad G;\ H \vdash \tau \preceq \tau' \qquad G;\ H \vdash \tau'}{G;\ H \vdash e : \tau'}\ \text{(T-SUB)}$$

Fig. 8. We define a set of stream typing rules incorporating the aforementioned temporal semantics. We use a subtyping rule (T-SUB) similar to that of Liquid Haskell [9, 34].

$\boxed{G;\ H \vdash \tau_1 \preceq \tau_2}$

$$\frac{G;\ H \vdash \forall x_1 : b\ .\ \varphi_1 \Rightarrow \varphi_2[x_2 \mapsto x_1]}{G;\ H \vdash \{x_1 : b \mid \varphi_1\} \preceq \{x_2 : b \mid \varphi_2\}}\ \text{(SUB-BASE)}$$

Fig. 9. Subtyping rule inspired by Liquid Haskell [9, 34], which we implement in MARVeLus. The arrow $\Rightarrow$ symbolizes logical implication.

$\boxed{G;\ H \vdash c}$

$$\frac{\mathsf{SMTValid}(c)}{G;\ \varnothing \vdash c}\ \text{(ENT-EMP)} \qquad \frac{G;\ H \vdash \forall x : b\ .\ \varphi[v \mapsto x] \Rightarrow c}{G;\ H, x : \{v : b \mid \varphi\} \vdash c}\ \text{(ENT-EXT)}$$

Fig. 10. We use similar entailment rules as those found in [34] for discharging verification conditions, where SMTValid() is a call to the SMT solver to check validity of a predicate.

$$H ::= \varnothing \mid H, x : \tau$$
$$G ::= \varnothing \mid G, f(x) : x : \tau_1 \longrightarrow \tau_2$$
$$\mathsf{hd}(\{w : b \mid \mathsf{hd}(\varphi_1) \wedge \bigcirc\varphi_2\}) \triangleq \{w : b \mid \mathsf{hd}(\varphi_1)\}$$
$$\mathsf{tl}(\{w : b \mid \mathsf{hd}(\varphi_1) \wedge \bigcirc\varphi_2\}) \triangleq \{w : b \mid \varphi_2\}$$
$$\mathsf{tl}(H) \triangleq \{\mathsf{tl}(H(x)) \mid x \in \mathsf{dom}(H)\}$$
$$\{w : b \mid \mathsf{hd}(\varphi_1) \wedge \bigcirc\varphi_2\} \preceq \mathsf{prev}(\{w : b \mid \varphi_2\}) \preceq \{w : b \mid \bigcirc\varphi_2\}$$
$$\mathsf{prev}(H) \triangleq \{\mathsf{prev}(H(x)) \mid x \in \mathsf{dom}(H)\}$$

Fig. 11. Typing Environments, and the operations on types and environments.

6.2 Environments, Types, and their Operations

Aside from the separation of function and stream environments, the two environments are defined in the standard way (Fig. 11). By virtue of our streams being constructed from initial values and subsequent steps (through fby), it is convenient to define operators hd() and tl() to extract the "immediate" and "suffix" of a type, respectively. For some type τ that can be written as $\{w : b \mid \mathsf{hd}(\varphi_1) \wedge \bigcirc\varphi_2\}$, an expression of type $\mathsf{hd}(\tau)$ is only required to satisfy the refinement predicate of τ in the current step. Hence, it only needs to satisfy $\mathsf{hd}(\varphi_1)$. Similarly, an expression of type $\mathsf{tl}(\tau)$ must satisfy, immediately, the refinement predicate that an expression of type τ will satisfy in the next time instant. The tl() operator is particularly useful for computing the types of rewritten expressions of the form $e \overset{v}{\hookrightarrow} e'$, and so we extend it to operate on the entire environment ($\mathsf{tl}(H)$). Analogously to the operational semantics, we must also capture a notion of "history" in the typing context. However, the result is not as exact. If σ represents a variable's past behavior, then H represents its future. As a variable is updated, its immediate future behavior specializes into a single value, which in the next time instant, gets added to σ. This process "consumes" the part of the refinement predicate that pertains to the current instant in preparation for determining the variable's future behavior. Unlike the case for σ, reversing this process in H is lossy, and the best we can accomplish is an approximation of the original type. Thus, if we started with a type $\{w : b \mid \mathsf{hd}(\varphi_1) \wedge \bigcirc\varphi_2\}$ and take its suffix $\mathsf{tl}(\{w : b \mid \mathsf{hd}(\varphi_1) \wedge \bigcirc\varphi_2\}) = \{w : b \mid \varphi_2\}$, we may not be able to recover $\mathsf{hd}(\varphi_1)$, at least not syntactically. However, we at least know that $\mathsf{prev}(\mathsf{tl}(\{w : b \mid \mathsf{hd}(\varphi_1) \wedge \bigcirc\varphi_2\}))$ is a subtype of $\{w : b \mid \bigcirc\varphi_2\}$. Using this, and potentially additional information brought in through invariants, we can approximate the type environment from the previous time instant, which may suffice even if we cannot recover the exact environment.

6.3 Summary of Typing Rules

(1) *Constants*: (T-CONST) encapsulates the behavior of refinement type synthesis for constants [34], assigning a specific singleton type to constants. Since constants do not change, we can strengthen the refinement predicate with $\Box$ to require that the constant c is typed as a stream that is always equal to c.
(2) *Variables*: (T-VAR) performs selfification on variables [34, 40], which strengthens the type by explicitly requiring that the refinement variable v is always equal to the term variable x. This is achieved by the addition of the $\Box(v = x)$ constraint in the refinement.
(3) *Stream Compositions*: (T-FBY-0) and (T-FBY-1) generate the verification conditions for typing the composition of two streams e_1 and e_2, with associated predicates $\mathsf{hd}(\psi_1)$ and ψ_2, respectively. Since the resultant stream must not change its base type, we require that e_1 and e_2 type

to the same base type. The fby construct allows one to combine values of both streams e_1 and e_2; naturally, the resultant stream inherits some properties of both. Crucially, the stream only takes the first value of e_1, after which it replicates the behavior of the stream e_2 but delayed by one cycle. Consequently, given e_1 satisfying property ψ_1 for at least the first cycle and e_2 satisfying ψ_2, the stream e_1 fby e_2 should satisfy both ψ_1 for the first cycle, and also $\bigcirc\psi_2$. The subscript denotes the value "buffered" by fby, which is empty in the first cycle (T-FBY-0), and may be occupied by a value in subsequent cycles (T-FBY-1). Since the first stream is only considered in the first cycle and is disregarded in determining the resultant type in (T-FBY-1), it is left unrefined. Instead, a stream's behavior with fby_u where $u \neq \text{nil}$ is determined by the u and e_2.

(4) *Local Bindings*: We define recursive (T-REC) and non-recursive (T-LET) local bindings as usual, noting that (T-REC) is crucial for proving inductive invariants when paired with (T-FBY). We note that T-LET requires type annotations be well-formed ($G;\ H \vdash \tau_1$), that is, the refinement predicate is a Boolean stream that only references in-scope variables. This is a direct lifting of the non-stream well-formedness judgement for other refinement type systems [34, 48]. Suppose we want to verify that the program `let rec x:{v:int | v >= 0} = 0 fby x + 1 in x` types to $\{v : \text{int} \mid v \geq 0\}$. Since x is recursively defined, verifying that all values of x are non-negative involves first verifying that the initial condition 0 and then the inductive definition x+1, with the assumption that $\square(x \geq 0)$ held in the previous instant. At first glance, the assumption $\square(x \geq 0)$ appears circular, essentially stating that the stream we want to check already satisfies the predicate we want to verify. However, we note that at each instant beyond the first time step, x represents the stream in the current time step, whereas $x + 1$ is the new definition that will take effect in the next time step. Therefore, the verification task shifts to proving that, assuming the stream x already satisfies $\square(x \geq 0)$, the predicate $\square((x + 1) \geq 0)$ is valid.

(5) *Branching Expressions*: As with other refinement type systems [34], the type to be checked is passed to each branch and further refined so that it only needs to be satisfied when the conditional of that branch is satisfied. This control-flow awareness prevents the type system from needlessly rejecting terms that the program will have never reached. This notion is lifted to stream types through the impl() operator, which inserts an implication statement at the state-predicate level, achieving the same effect as in non-stream refinement type systems with a similar rule.

(6) *Function Application*: Since functions are imported from a non-stream language and applied point-wise to the values of stream variables, they are considered constant and thus their types are unchanging over time. As a result, the argument variable should type to the function's argument type at all times. By extension, the return type of the function application should be enforced for all time. This is encapsulated by the (T-APP) rule, which augments the original (non-stream) function type with the LTL $\square$ operator to require that these properties hold over all time.

(7) *Robot-specific commands*: Due to the unpredictability of real-world physical parameters, we exclude robot_get and robot_store from refinement type-checking, and treat them as functions that simply return the appropriate un-refined term for the base Zélus type checker. Instead, we require that each call to robot_get be accompanied by models, so that each variable intended to derive values from the real world has a deterministic verifiable model. Indeed, the user is responsible for ensuring that the model does in fact reflect expected real-world conditions, which are impossible to predict at compile-time. Since the models keyword is intended to be semantically transparent, it directly inherits the type of its inner expression.

Although it is outside the control of MARVeLus, it is assumed that the robot-specific command following models has the same type.

(8) *Typing* nil: Since nil can be pushed onto the history of any variable, we allow it to type to anything. While this may appear disconcerting, the static initialization and causality analyses in Zélus [7] ensure that well-formed programs do not allow nilto be executed. That is, a variable that does not have immediate self-dependencies or have circular dependencies with other variables will never access a variable whose history ends in nil.

(9) *Subtyping*: We use the subtyping rule from Liquid Haskell [34, 48], which stipulates that an expression which already types to τ may also type to τ' if τ' is a subtype of τ. This allows refinement types to be checked without needing an exact match to the user-supplied type. We use the standard well-formedness rules surrouding type refinements to ensure τ', which may be user-supplied, has a well-typed predicate and does not contain unbound variables. We also adopt the subtyping judgement $G; H \vdash \tau_1 \preceq \tau_2$ with rule (SUB-BASE) (Fig. 9 to subtype refined base types in a similar style as Liquid Haskell. A function subtyping rule is not necessary as MARVeLus streams do not support higher-order functions. We use a similar entailment judgement $G; H \vdash c$ (Fig. 10) which provides the interface to the SMT solver by constructing the verification condition to be checked for validity given the subtyping implication and refinement predicates bound to variables in the typing environment.

6.4 Type Safety

We prove type safety roughly following the classical syntax-guided manner of Wright and Felleisen [55]. A key difference arises in our type system from the decoupling of program terms from the values they produce in the synchronous semantics. Type safety must apply not only to the values emitted, but also to the subsequent evaluations of the stream term to ensure that the program remains safe in the future. We also note that existing work by Boulmé and Hamon [10] has explored synchronous properties of languages such as Lustre, some of which resemble progress in conventional languages. We begin by defining convenience operators basetype and now, which "un-refine" a refined type and retrieve the most recent entry of a variable's history, respectively:

$$\mathsf{basetype}(\{w : b \mid \varphi\}) = b \qquad\qquad \mathsf{now}(h :: v) = v$$

We also define conditions under which the function and stream term environments correspond with their type environment counterparts. Namely, we want to enforce some notion of well-typedness on the environments:

Definition 1 (Function Environment Correspondence). $G : \mathsf{name} \to T$ ***corresponds with*** $S : \mathsf{name} \to (x : \tau_1 \to \tau_2)$ *when the following conditions are met:*

- $\mathsf{dom}(G) \subseteq \mathsf{dom}(S)$
- $\forall f \in \mathsf{dom}(G), G; H \vdash S(f) : G(x)$

Definition 2 (Stream Environment Correspondence). $H : \mathsf{name} \to T$ ***corresponds with*** $\sigma : \mathsf{name} \to h$ *when the following conditions are met:*

- $\mathsf{dom}(H) \subseteq \mathsf{dom}(\sigma)$
- $\forall x \in \mathsf{dom}(H)$*:*
  - $G; H \vdash \sigma(x) : \mathsf{basetype}(H(x))$ *and*
  - $G; H \vdash \mathsf{now}(\sigma(x)) : \mathsf{hd}(H(x))$
- $\mathsf{prev}(\sigma)$ *corresponds with any* $\mathsf{prev}(H)$

Definition 1 is standard. We note the absence of H in its typing judgement as functions are judged in their original non-stream contexts using the standard typing judgement. Definition

2 involves streams and thus requires different treatment. Stream types determine a variable's current and expected future trajectory, while stream histories record a variable's past and current trajectory. They intersect at the present moment, where both the history and type of a variable refer to the variable's current state. Thus, at any given moment, we can only make a precise, refinement typing judgement about the variable's current value. Since histories are initialized to be empty and are extended only by let-bindings (recursive or otherwise), we can ensure that let-bindings maintain correspondence between the type and term environments for all time. This is proven explicitly in the (T-LETREC) case in our type preservation proof, and assumed elsewhere. Because the prev() operator is unable to exactly recover the original refinement of a type, we simply require correspondence of $\mathrm{prev}(\sigma)$ with any $\mathrm{prev}(H)$, noting that for any H which evolves into H', $\mathrm{prev}(H')$ will necessarily produce subtypes of variables in H. We can however ascertain that a variable corresponds with its base type at any point in time.

Theorem 1 (Type Safety). *If H corresponds with σ and G corresponds with S, and if $G; H \vdash e : \tau$, then $\exists e'$ such that $S; \sigma \vdash e \overset{v}{\hookrightarrow} e'$ where v is a value, $G, H \vdash v : \mathsf{hd}(\tau)$, and $G, \mathsf{tl}(H) \vdash e' : \mathsf{tl}(\tau)$*

Proof. By proving progress and preservation on well-typed terms. □

6.4.1 Progress. The standard notion of progress stipulates that a well-typed program term is either a value or can undergo reduction via the semantic rules. Since all MARVeLus streams are designed to emit values in perpetuity, including constant streams, we must show that there is always a rule that allows the program to take another semantic step, and thus emit another value.

Lemma 2 (Progress). *If H corresponds with σ and G corresponds with S, if $G; H \vdash e : \tau$, then $\exists e'$ such that $S; \sigma \vdash e \overset{v}{\hookrightarrow} e'$ where v is a value.*

Proof. By induction over the typing derivation of e. Full proof in the extended paper [18]. □

6.4.2 Preservation. When a stream expression e is evaluated, it not only produces a value v representing its immediate state at this instant, but also becomes e', representing the stream to be evaluated in the next instant. Over the course of several evaluations, the stream that was once e becomes "realized" as a series of emitted values $v_0,\ v_1,\ v_2 \ldots$ and stream definitions $e', e'', e''' \ldots$. Combined with the fact that refinement predicates are considered in the time instant in which a stream is evaluated, this means that a predicate that accurately describes a stream in one instant may no longer be satisfied over subsequent evaluations. Consider the stream s whose next values will be $[-1, 0, 1, 2, 3 \ldots]$. The predicate $\varphi = (s < 0) \wedge \bigcirc\Box(s \geq 0)$ accurately describes s in the current time instant. However, after s emits a value and becomes s', whose next values are $[0, 1, 2, 3 \ldots]$, φ no longer accurately describes s', as the value it emits next is not negative. The state predicate $s < 0$ within φ should only apply to the first state of s. An implication that arises is that type refinements also need to evolve over time to accurately describe stream behavior, a phenomenon that is not normally seen in refinement type systems. This somewhat complicates proofs of type safety, as stream expressions that step do not necessarily continue to inhabit their original types.

However, we note that a stream e' produced by evaluating e is a "suffix" of e; in other words, the values emitted by e' are identical to those emitted by e after the first value. Likewise, if e satisfies a predicate φ, then e' will satisfy its "suffix". This allows us to relate the types that e and e' inhabit and derive a modified notion of type safety, even if their types do not exactly match. If a predicate φ can be written as $\mathsf{hd}(\varphi_1) \wedge \bigcirc\varphi_2$ for some secondary predicates φ_1 and φ_2, then its suffix is φ_2. Thus, if e satisfies φ, and in the current context $e \overset{v}{\hookrightarrow} e'$, then v satisfies $\mathsf{hd}(\varphi_1)$ and e' satisfies φ_2. From this, we can derive types such that if $G;\ H \vdash e : \{w : b \mid \mathsf{hd}(\varphi_1) \wedge \bigcirc\varphi_2\}$, then

$G;\ H \vdash v : \{w : b \mid \mathsf{hd}(\varphi_1)\}$ and $G;\ \mathsf{tl}(H) \vdash e' : \{w : b \mid \varphi_2\}$ where $\mathsf{tl}(H)$ is the immediate successor of H:

$$\mathsf{dom}(\mathsf{tl}(H)) = \mathsf{dom}(H),\ \forall\, x \in \mathsf{dom}(\mathsf{tl}(H))\ .\ (\mathsf{tl}(H)(x)) = \mathsf{tl}(H(x))$$

where for a refined type $\tau = \{w : b \mid \mathsf{hd}(\varphi_1) \wedge \bigcirc\varphi_2\}$, $\mathsf{tl}(\tau) = \{w : b \mid \varphi_2\}$. We prove that φ_1 and φ_2 exist for any φ constructed in our specification grammar:

Lemma 3 (Temporal Predicate Expansion). *Let ψ be a temporal predicate. Then there exist $\psi_1,\ \psi_2$ such that $\psi \equiv \mathsf{hd}(\psi_1) \wedge \bigcirc\psi_2$*

Proof. By induction on the specification grammar. Full proof in Appendix A. □

Lemma 4 (Type Preservation). *Let e be a synchronous stream expression. If H corresponds with σ and G corresponds with S, then if $G, H \vdash e : \tau$ and $S;\ \sigma \vdash e \overset{v}{\hookrightarrow} e'$, then $G, H \vdash v : \mathsf{hd}(\tau)$ and $G, \mathsf{tl}(H) \vdash e' : \mathsf{tl}(\tau)$*

Proof. By structural induction on the operational semantics. Full proof in Appendix A. □

The interpretation of this theorem is that, if some expression satisfies a property, then evaluating that expression results in a value that satisfies that property for the single time instant in which it is used, and the remaining rewritten expression will continue to satisfy the remaining "suffix" of the property. Note that e' is typed in a modified environment $\mathsf{tl}(H)$, which expresses the notion of variables updating across time instants. We prove this theorem holds in our type system via induction on the operational semantics derivation (Appendix A).

7 Implementation and Verification of a Robotics Application

The increasing automation of automobiles and other modes of transport sees a corresponding increase in the responsibilities of software for the safety of passengers, cargo, and infrastructure [2, 44]. As safety-critical CPS, transportation automation is a fitting application for MARVeLus. We develop our example around vehicle collision avoidance, which is responsible for maintaining safe separation of the controlled "ego" vehicle from other vehicles in the environment. We base our model partially on existing work formally verifying collision avoidance for adaptive cruise control in autonomous vehicles [20, 39], with the primary difference being the use of a discretized model for two moving vehicles.

We make the simplifying assumption that the vehicles involved move along a single one-dimensional path, as is the case for autonomous vehicles with existing lane-keeping, guided buses, or rail vehicles. Thus, the ego vehicle maintains separation by controlling its acceleration only. We also assume that the ego vehicle is traveling behind another vehicle, which is the only other participant in the scenario. The other (henceforth "leader") vehicle is free to take arbitrary actions, bounded only by acceleration limits. Besides externally observable properties (such as velocity and position), the ego vehicle has no *a priori* knowledge of the leader's actions. The property we verify is that collision never occurs. Given that the ego vehicle starts behind the leader, this property can be expressed as $\Box(x^f < x^\ell)$; that is, the ego vehicle position x^f never exceeds the leader's position x^ℓ. Since we are primarily interested in verifying the control logic, we assume the ego vehicle has perfect knowledge of its own position and velocity, as well as that of the leader, and the vehicle instantaneously attains the commanded acceleration.

In our deterministic simulation, we implement the "worst-case" situation, in which the ego vehicle is accelerating at its maximum allowable rate while the leader suddenly brakes with maximum force until it completely stops. This ensures that the ego vehicle will also react safely to the less aggressive maneuvers that it is more likely to encounter in normal operation.

$$
\begin{array}{ll}
x_0^\ell = x_{init}^\ell & x_{n+1}^\ell = x_n^\ell + v_n^\ell \cdot dt \\
v_0^\ell = v_{init} & v_{n+1}^\ell = \max(0, v_n^\ell + a_n^\ell \cdot dt) \\
a_0^\ell = 0 & a_{n+1}^\ell = \begin{cases} -b & \text{if } (x_n^\ell \geq x_{brake}) \\ 0 & \text{otherwise} \end{cases} \\
x_0^f = 0 & x_{n+1}^f = x_n^f + v_n^f \cdot dt \\
v_0^f = v_{init} & v_{n+1}^f = \max(0, v_n^f + a_n^f \cdot dt) \\
a_0^f = a_{max} & a_{n+1}^f = \begin{cases} -b & \text{if } (x_n^f + s_n^f) \geq \left(x_n^\ell + \frac{(v_n^\ell - b \cdot dt)^2}{2b} + \frac{(v_n^\ell - b \cdot dt) \cdot dt}{2} \right) \\ a_{max} & \text{otherwise} \end{cases} \\
 & s_n^f = \frac{(v_n^f)^2}{2b} + (1 + \frac{a_{max}}{b})(2v_n^f \cdot dt + \frac{4a_{max} \cdot dt^2}{2}) + \frac{v_n^f \cdot dt}{2} \\
\forall n \,.\, x_n^f < x_n^\ell &
\end{array}
$$

Fig. 12. Dynamics of the discrete-time collision avoidance system. x^f and x^ℓ are the positions of the ego and leader vehicles, v^ℓ and v^f are their velocities, a^ℓ and a^f their commanded accelerations, and s^f the minimum stopping distance of the ego vehicle if it were to brake at the current instant. The safety condition specifies that no collision between the vehicles at any instant.

We model the collision-avoidance scenario as a discrete-time system (Fig. 12), in which the leader and follower dynamics evolve at a constant time step of dt seconds. At each time step, vehicle position and velocity are integrated using the standard kinematic equations of motion, while their commanded accelerations are determined by the vehicle's controller in response to the updated velocity and position values. A positive acceleration command represents the vehicle applying the throttle and thus accelerating, while a negative command represents braking at that acceleration value. The ego vehicle initially accelerates at the maximum possible rate a_{max} and commands full braking acceleration $-b$ when it is no longer safe to continue accelerating as determined by the vehicles' positions and relative velocity. The leader vehicle coasts at its starting velocity for a predetermined distance, after which it commands an acceleration $-b$ which causes it to brake with maximum force until it comes to a complete stop. The ego vehicle has no knowledge of precisely when the leader will brake, and must ensure it always maintains sufficient separation to safely stop at any time. This is encoded into its controls as the relation $(x_n^f + s_n^f) \geq \left(x_n^\ell + \frac{(v_n^\ell - b \cdot dt)^2}{2b} + \frac{(v_n^\ell - b \cdot dt) \cdot dt}{2} \right)$, which uses the ego vehicle's anticipated stopping distance under current conditions and maximum braking s^f, and the assumption that the leader has already begun braking, to determine their final positions. If the ego's anticipated position reaches or exceeds that of the leader's, then it brakes until this condition no longer holds. We constrain velocities to be non-negative, to model vehicles stopping rather than accelerating backwards, when a negative acceleration is commanded while stationary. We note that the derivation of s^f, including its discretization adjustment, has been covered in existing work [20, 36].

The formally-verified program consists of 89 lines of code, of which 52 are executable code (44 for core functionality and 8 for robot interfaces and logging) and 37 are additional lines for type specifications. The verified source code can be found in Appendix B. Verification on a 2019 MacBook Pro with a 1.4 GHz Intel Core i5-8257U processor and 16 GB of memory typically takes less than 1.3 seconds.

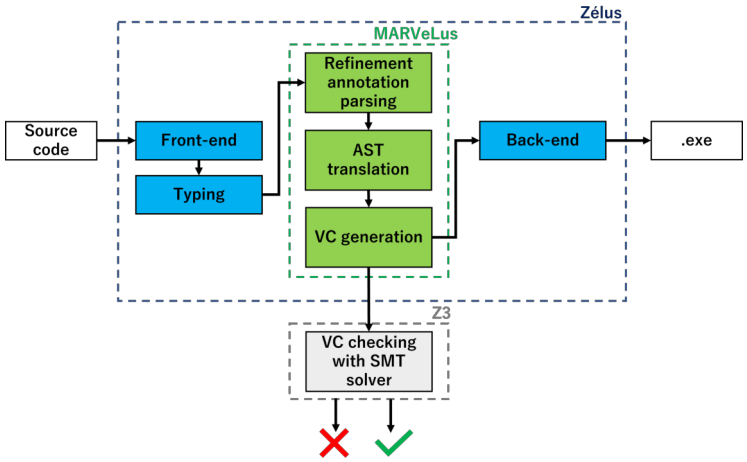


Fig. 13. MARVeLus compilation steps

8 Language Implementation

The refinement type verification is implemented as an additional pass in the Zélus compiler. This step follows the type checking, which is responsible for validating the base-types used in the program. The verification algorithm is built using Z3[24], an SMT-solver that provides the support for proving logical propositions and searching for counterexamples.

MARVeLus extends the Zélus Compiler with refinement type checking for the source code. This additional step identifies refinement type annotation in the source program, translates the Zélus Abstract Syntax Tree (AST) to an AST with Z3 data structures, generates verification conditions (VCs) and sends VCs to the Z3 SMT solver to check for satisfiability. The compilation steps and interfaces between Zélus and MARVeLus are illustrated in Fig. 13. An archive of the source code is available in the non-anonymized supplementary materials.

8.1 Generation of Verification Conditions

The refinement type pass in the compiler translates the Zélus AST enhanced with refinement types to a Z3 AST. In this translation, the program expressions, operations, patterns and equations are converted into Z3 expression objects. These objects are used to construct boolean expressions that are used as verification conditions.

The verification conditions are generated following the typing rules defined in Fig. 8. Each verification condition's satisfiability is checked using the Z3 SMT solver. Conditions proven to be true become powerful statements when generating other verification conditions in later parts of the program.

8.2 Verification of Refinement Variables

To verify a refinement typed variable declared in the form $var : \{w : b \mid \varphi\} = e$, the conjunction of all expressions, φ_i, in the current environment, E, must imply that the refinement φ is also true, $\bigwedge^{E} \varphi_i \longrightarrow \varphi$. This arises from the (ENT-EXT) entailment rule (Fig. 10) and the need to check that refinements assumed in the environment are consistent, so that the implication is not vacuously true. From the variable declaration, the expression $var = e$ is added to the expression environment as an assumption.

An SMT Solver checks satisfiability by finding an example that makes the expression true. However, our goal is to prove that a verification condition is always satisfied and we need to rearrange the verification condition so the Z3 solver cannot find examples that make the verification condition false. This is achieved by the negation of the initial expression, resulting in the following verification condition: $\neg(\bigwedge^{E} \varphi_i \rightarrow \varphi)$. If this verification condition is satisfied, the refinement type is not valid and the program shows the counter-examples to the user. On the other hand, if the verification condition is unsatisfiable, the refinement expression is valid and the refinement type assumption is added to the expression environment.

```
1     let pi = 3.14159 in
2     let w = 2.*.pi in
3     let y: { v : float | v >= pi} = 4.0 in y
```

In the above example, `pi` is a constant variable, `w` is declared in terms of `pi`, and `y` is annotated with a refinement for which Z3 must check satisfiability. The expression environment, E, after variable y is declared is

$$E = (pi = 3.14159) \wedge (w = 2 * pi) \wedge (y = 4.0)$$

Where the last expression corresponds to the assignment for the newly declared refinement-type variable. To show the validity of the refinement predicate, $\varphi \equiv (y >= pi)$, given the variable assignment, the following verification condition is generated:

$$\neg((pi = 3.14159) \wedge (w = 2 * pi) \wedge (y = 4.0) \rightarrow (y \geq pi))$$

This expression is checked to be unsatisfiable by Z3, since $4 \geq 3.14159$. Therefore, the refinement type is valid and the condition $y \geq pi$ is then added as an assumption to be used in further parts of the program.

8.3 Verification of Refinement Functions

A refinement function can be annotated with refined input arguments and a refined return type. In the example shown below, f is a unary refinement function of argument $\{v : int \mid v < 0\}$ and return type $\{v : int \mid v >= 0\}$. The steps to prove the refinement return type are similar to proving refinement types for variables.

In this proof, a local expression environment is created to store function body expressions and refinement-type information about the function arguments. Once the local environment is built, the function return type is used to build the verification condition that will be checked by Z3.

If the function return type is proven to be always true, it is added to a function environment. The function environment is used to check that during a function call, the arguments follow the constraints specified during the function definition.

Once proved the function application to its argument is represented by an uninterpreted function, which is supported by Z3.

```
1  let f (x: { v : float | v < 0.}) : { v : float | v >= 0. } =
2      let y = x *. x in
3      y
```

In the above example, function f only accepts negative float inputs and the program ensures that the output is positive. The local expression environment after function declaration and before the function proof is

$$E = (x < 0) \wedge (y = x * x) \wedge (v = y)$$

Where the first condition is created from the argument refinement-type definition, the second condition is created from the function body and the third condition is a binding from the function

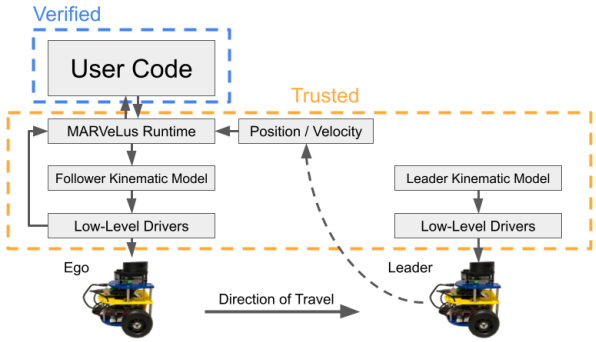


Fig. 14. System Architecture for the Collision Avoidance Demonstrator

return type to the function body return variable. For this example, Z3 checks if the verification condition $v \geq 0$ is satisfiable using the following verification condition:

$$\neg((x < 0) \wedge (y = x * x) \wedge (v = y) \rightarrow (v \geq 0))$$

9 Experimental Methods

In addition to the aforementioned language extensions, we also made enhancements to the MARVeLus robot runtime [20] to support multi-robot systems. We found that the original MARVeLus robotics extensions were not easily scalable beyond a single robot, with unpredictable latency being especially problematic for synchronizing multiple robots. We retain the original software architecture, with separate code for the user MARVeLus program and runtime, the robots' kinematic models simulating the vehicles' physical attributes, and low-level drivers for sensors, actuators, and networking (Fig. 14). These components all run locally on each robot. We re-implemented the communications stack to accept synchronization commands from a centralized server, ensuring timing consistency between experiments. We also selected the M-Bot Mini as our primary development target. The robots run along an 8 m track (Fig. 15). The trusted software base of our experiment consists of the MARVeLus compiler, runtime, kinematic models, and sensor and low-level drivers. That is, we trust that sensor data is always available and valid, and that actuators always attain their commanded values. We also trust that communications is instantaneous and error-free, and that data stays sufficiently consistent system-wide. Our verified control code runs on the follower vehicle, while the leader vehicle acts independently. The robots are equipped with wheel encoders to measure distance traveled and velocity, and the leader vehicle reports its position and velocity to the ego, to simulate the direct sensing that would in practice be performed using computer vision or LiDAR. The leader and ego vehicles do not otherwise communicate with one another.

In addition to the collision avoidance scenario described in Section 7, in which the leader brakes abruptly and stops, we tested two other variations: (1) The leader remains stationary throughout, simulating an obstacle present on the track or roadway. (2) The leader decelerates to a constant slow speed, but does not fully stop, allowing the ego vehicle to continue moving forward as long as it maintains safe separation. When both robots have stopped, or after a predetermined period of time in the case of the slow-moving leader, the final distance between the two robots is recorded.

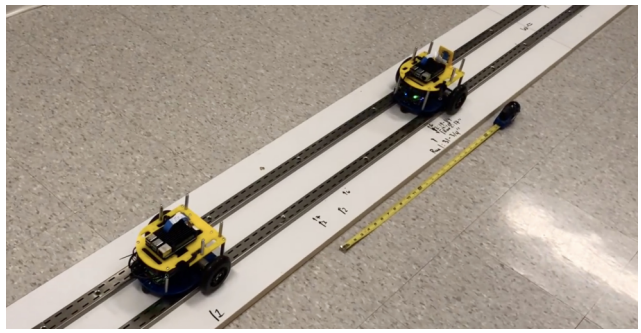


Fig. 15. Ego and Leader vehicles traveling on rails

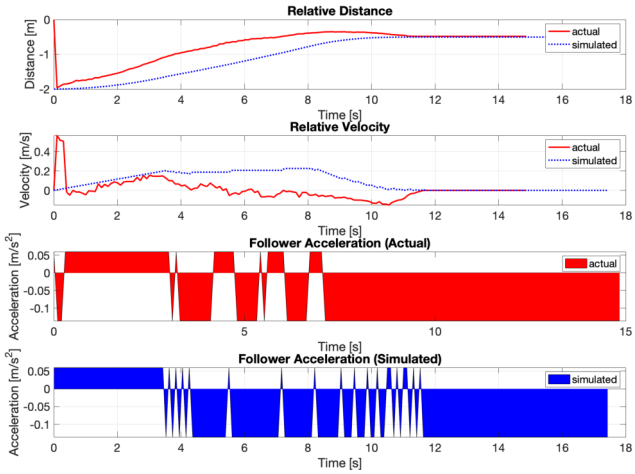


Fig. 16. Relative kinematic plot for actual and simulated experiments

10 Results

We first ran simulations of the experiments to obtain a basis of comparison for the experimental results. We then conducted ten trials for each of the three collision avoidance scenarios described in Section 9. The data obtained from one trial of the collision avoidance scenario in which the leader brakes abruptly and comes to a complete stop, is plotted in Fig. 16. Data and plots from the other experiments are included in the appendix.

The kinematic data from the actual experiment differ somewhat from their simulated counterparts, which we believe arose from subtle differences in timing, initial conditions, or physical aspects of the real robots which were not accounted for in our original model. The actual commanded accelerations differed significantly from the simulation values, which is to be expected as they are reacting to different positions and velocities at any given moment. However, in spite of the variations in acceleration, the follower vehicle maintains a safe distance from the leader vehicle, allowing for the actual distance and velocity curves to converge to the simulated curves towards the end of the experiment. After finishing each of the 10 trials in each scenario, the distance between

the leader and follower vehicle in their final positions was manually measured and recorded. The sample statistics for these collected distances can be seen in Table 1.

Table 1. Summary statistics of final distance measurements between vehicles

Statistic	Leader braking	Leader decelerating only	Leader stationary
Mean	0.4610 m	1.0135 m	0.4826 m
Standard Deviation	0.06 m	0.4902 m	0.0910 m
Minimum	0.3429 m	0.7049 m	0.2794 m
Maximum	0.5207 m	2.1082 m	0.5525 m

11 Related Work

11.1 Verification of CPS

Hagen and Tinelli [31] developed the Kind prover for Lustre, which introduces SMT-based Lustre verification using k-induction, in order to automate safety proofs on synchronous dataflow programs. As with Kind, MARVeLus leverages SMT solving to inductively prove properties on synchronous programs. Although MARVeLus and Kind take different approaches to verification, a combined approach may lead to rigorous inductive proofs with reduced effort in finding invariants.

Champion et al. [17] introduce Kind 2, an SMT-based model checker that produces invariants for synchronous programs over a bounded number of steps. For any successful proof, Kind 2 generates a proof certificate in a k-induction invariant form that can be validated by other solvers. In contrast to model-checking approach adopted by Kind 2, MARVeLus uses type properties directly to verify programs, though the end goal of reasoning on invariants remains similar.

Kamburjan [35] presents an object-oriented approach to verifying invariants in hybrid CPS. Similarly to MARVeLus, the rely-guarantee principle resembles our use of inductive invariants in discrete models, but extends this to the continuous space as well. To our knowledge, the work does not yet appear to support real-time execution of verified source code.

11.2 Verification via Typing

Languages such as F* [51] and Liquid Haskell [34] demonstrate the utility of allowing types to be augmented with functional or logical annotations. Both F* and Liquid Haskell interpret type annotations as constraints which are used to generate verification conditions for SMT solvers, such as Z3 [24]. Many of the MARVeLus type system components take cues from these two languages, lifting them into the space of synchronous programming. We chose to adopt refinement types as our theory of choice; although it is more constrained than dependent types, the quantifier-free nature of refinement predicates allows for SMT-decidable verification conditions and thus predictability [34]. A related application of verification via typing was demonstrated with Π4, a dependently-typed language for networking hardware.[25]

11.3 Verification and Execution

Aside from MARVeLus, VeriPhy [8] and Koord [29] also combine formal verification with execution on real systems. Although the goals of MARVeLus and Koord are similar, they differ in that Koord is intended primarily for higher-level coordination between robots, while MARVeLus is more concerned with the lower-level controls that may be specific to individual robots [29]. VeriPhy [8] and similarly Melecha et al. [37] link a high-level abstract model of CPS to a lower-level implementation, either through runtime monitoring and switching, or verified translation.

11.4 Testing Autonomous Vehicles at Small-Scale

Various experiments have previously been done on small-scale vehicles to simulate real-world autonomous vehicles [49, 50, 54]. Rupp et al. [49] simulated autonomous vehicle maneuvers using scaled-down trucks and passenger cars equipped with sensors, actuators, cameras, and communication devices. ACC was implemented where two vehicles followed a lead vehicle with a constant distance or time gap while the lead vehicle tracks a given velocity profile. The experiments were done both with and without car-to-car communication. Position measurement in the platooning experiments was done visually using fiducial markers, as opposed to our simulated distance sensing.

12 Future Work

12.1 Invariants

In its current implementation, MARVeLus simply checks typing constraints supplied by the user, relying on the user to assess the feasibility of counterexamples to refine the constraints with invariants. During program verification, one may encounter spurious counterexamples, in which an unsafe program state is found, but is not feasible in an actual system. These may be countered by supplying the verifier with invariants, as we have done manually in our example programs. In general, finding the necessary relations is not immediately obvious and requires significant additional effort by the user. There exist works that automate the process of inferring invariants [28], including works that find invariants in synchronous programs [17]. Additionally, Liquid Haskell supports refinement inference through Horn constraint solving [34], which may be adapted to take advantage of synchronous programming properties. Although orthogonal to the objectives of the current work, supporting invariant generation would help to reduce the effort involved in refining program constraints and improve user experience, particularly for complex programs.

12.2 Trusted Components

Although developing a single language for verification and execution holds the promise of a compact trusted base, the current implementation makes several assumptions about the safety of components interacting with the user's verified MARVeLus code. Firstly, we assume that the compiler produces a safe executable from the verified code. Formally-verified compilation of synchronous programs has been explored in other works [11], which allows one to directly produce executables that provably reflect their specified synchronous behavior. Combining our verification methods with a formally-verified compiler would establish a certified chain of trust in the compilation process.

Beyond compilation, we also trust the safety of MARVeLus runtime layer. For instance, we trust that network latency and throughput can be disregarded for the performance requirements of our runtime. However, this may not always be the case in real applications, particularly for resource-constrained embedded systems where the communications overhead may be nontrivial. Thus, it is necessary to characterize and verify the message-passing protocol employed for communications between the various robot components, of which there is existing work [3].

We also note that, for simplicity, we chose to write the low-level control code in C, which includes feedback control for robot speed and heading. This choice was intentional, as we chose to elide some of the lower-level implementation details from our verified code example in favor of clarity. However, we note that certain components, such as motor speed control and velocity sensing, may be implemented in verified MARVeLus as well.

For CPS operating in the real world, trust goes beyond verifying the "cyber" components alone. Compile-time verification implicitly assumes that system or environment assumptions hold for the real system. Error sources such as noise or external perturbations that are not properly modeled may put the system into an unsafe condition not accounted for in the specification and verification

steps. There is currently no way to enforce these assumptions at runtime in MARVeLus. Runtime monitoring, perhaps based on the verified invariants in MARVeLus code could help to mitigate or notify the system of anomalies [5, 8]. Additionally, one may wish to obtain robustness measures on verification results, to determine the system's immunity to disturbances [26].

12.3 Richer Specifications

Along with simulation and modeling of discrete systems, Zélus supports hybrid systems defined by differential equations, zero crossings, and labeled automata [12]. Presently, MARVeLus covers a subset of the discrete dynamics. Expanding the type theory to cover hybrid behavior would allow MARVeLus programs to be applied to a much greater range of CPS applications and provide more realistic models. This may be possible through the use of differential invariants as in dL [43] or barrier certificates [46] The current specification grammar can also be expanded to include other temporal operators, leading to other properties outside of invariance to be considered as well, and in fact there is existing work exploring the semantics of temporal logic in hybrid systems [33]. Finally, we currently disregard numerical precision and treat most numerical values as floating-point numbers, which are interpreted internally as reals. However, this can be imprecise, especially on embedded systems that have limited support for high-precision floating-point numbers. Some research is also present in the intersection of probabilistic and synchronous programming, such as ProbZelus [4], which may enable specification and analysis of stochastic processes.

13 Conclusion

MARVeLus provides a method for combining refinement types with synchronous programming to formally verify cyber-physical systems directly in the source language. Since programming languages typically used for CPS verification and modeling may not be expressive enough for execution, and languages typically used to implement executable systems may not have well-defined formal semantics, it is not always clear whether a CPS implementation accurately reflects its verified counterpart. Synchronous languages are well-suited for modeling embedded systems. Thus, a verifiable and executable language gives CPS designers increased confidence in developing safety-critical systems. Part of this confidence comes from the certainty that the verification results produced are indeed correct. The extended semantics and type system we develop in this paper, along with the formal proof of type safety built upon these extensions of MARVeLus, paves the way towards building this certainty in the language. By simplifying the process of obtaining rigorous safety guarantees in CPS and providing a unified platform in which one can both verify and implement CPS, MARVeLus provides an end-to-end solution for accelerating the development of safety-critical systems and lowering the barrier for entry to formal verification.

Data Availability Statement

Experiment data, source code, and an executable artifact are available on Zenodo [19].

Acknowledgments

We warmly thank Marc Pouzet and Timothy Bourke for interesting discussions, and for sharing their expertise of synchronous languages, as well as Ranjit Jhala for his detailed explanations of refinement types. We thank Tanner Slagel, Lauren White, Laura Titolo, Aaron Dutle and César Muñoz from the NASA Langley Formal Methods team, for their comments on earlier versions of this paper. We thank Peter Gaskell and the ROB 550 course staff for providing the M-Bot Mini robots used in our experiments, as well as Tigist Shiferaw and Serra Dane for their help with the experiments. Finally, we thank the reviewers for their thorough feedback and insightful comments. This research was supported in part by NSF Grant CCF-2348706.

A Omitted Proofs

A.1 Proof of Lemma 3 (Predicate Expansion)

Proof. By induction on the specification grammar (Fig. 3)
Case p: $\Psi \equiv p$ (where p is a state predicate): then $\Psi_1 \equiv p$ and $\Psi_2 \equiv \top$ satisfy the desired condition.
Case $\wedge$: $\Psi \equiv \psi_l \wedge \psi_r$ (where $\wedge$ is a binary logical connective): by IH, $\exists \psi_{l,1},\ \psi_{l,2}$ s.t. $\psi_l \equiv \mathsf{hd}(\psi_{l,1}) \wedge \bigcirc\psi_{l,2}$ and likewise $\exists \psi_{r,1},\ \psi_{r,2}$ s.t. $\psi_r \equiv \mathsf{hd}(\psi_{r,1}) \wedge \bigcirc\psi_{r,2}$. Then $\Psi_1 \equiv \mathsf{hd}(\psi_{l,1} \wedge \psi_{r,1})$ and $\Psi_2 \equiv (\psi_{l,2} \wedge \psi_{r,2})$ satisfy the desired condition.
Case $\bigcirc$: $\Psi \equiv \bigcirc\psi$, therefore $\Psi_1 \equiv \top$ and $\Psi_2 \equiv \psi$ satisfy the desired condition.
Case $\Box$: We have $\Psi \equiv \Box\psi$. By IH, $\exists \psi_1,\ \psi_2$ s.t. $\psi \equiv \mathsf{hd}(\psi_1) \wedge \bigcirc\psi_2$. Note that $\Box\psi \equiv \psi \wedge \bigcirc\Box\psi$. Therefore $\Box\psi \equiv (\mathsf{hd}(\psi_1) \wedge \bigcirc\psi_2) \wedge \bigcirc\Box\psi \equiv \mathsf{hd}(\psi_1) \wedge \bigcirc(\psi_2 \wedge \Box\psi)$. Then $\Psi_1 \equiv \psi_1$ and $\Psi_2 \equiv \psi_2 \wedge \Box\psi$ satisfy the desired condition.

□

A.2 Proof of Lemma 4 (Preservation)

Proof. By structural induction on the operational semantics derivation. We prove by cases on the last rule used.
Case S-LETREC: $\mathsf{let\ rec}_h\ x : \tau = e_1\ \mathsf{in}\ e_2$

- Assume $G;\ H \vdash \mathsf{let\ rec}_h\ x : \tau = e_1\ \mathsf{in}\ e_2 : \tau$ and $S;\ \sigma \vdash \mathsf{let\ rec}_h\ x : \tau = e_1\ \mathsf{in}\ e_2 \overset{w}{\hookrightarrow} \mathsf{let\ rec}_{h::v}\ x : \mathsf{tl}(\tau_1) = e_1'\ \mathsf{in}\ e_2'$.
- Furthermore, we prove that h consists solely of well-typed values. Specifically, we show that, after evaluation of (S-LETREC), $G;\ H \vdash v : \mathsf{hd}(\tau_1)$ where v is the last value in h. This ensures that σ stays consistent with H
- Induct on the number of elements in h.
- Suppose $h = []$.
  - By assumption, $S;\ \sigma, x = [\mathsf{nil}] \vdash e_1 \overset{v}{\hookrightarrow} e_1'$.
  - By assumption and the above, $S;\ \sigma, x = [v] \vdash e_2 \overset{w}{\hookrightarrow} e_2'$.
  - T-LETREC must be the last (non-subtype [9]) rule used in the typing derivation. By inversion:
    * $G;\ H, x : \tau_1 \vdash e_1 : \tau_1$
    * $G;\ H, x : \tau_1 \vdash e_2 : \tau$
  - Since τ_1 is well-formed ($G;\ H \vdash \tau_1$) and $\mathsf{dom}(H) = \mathsf{dom}(\mathsf{tl}(H))$, $\mathsf{tl}(\tau_1)$ is also well-formed, i.e. $G;\ \mathsf{tl}(H) \vdash \mathsf{tl}(\tau_1)$
  - By the Induction Hypothesis and Lemma 3:
    * $G;\ H, x : \tau_1 \vdash v : \mathsf{hd}(\tau_1)$. Thus v is well-typed.
    * $G;\ \mathsf{tl}(H), x : \mathsf{tl}(\tau_1) \vdash e_1' : \mathsf{tl}(\tau_1)$
    * $G;\ H, x : \tau_1 \vdash w : \mathsf{hd}(\tau)$
    * $G;\ \mathsf{tl}(H), x : \mathsf{tl}(\tau_1) \vdash e_2' : \mathsf{tl}(\tau)$
  - By (T-FBY), $G;\ \mathsf{tl}(H) \vdash \mathsf{let\ rec}_v\ x : \mathsf{tl}(\tau_1) = e_1'\ \mathsf{in}\ e_2' : \mathsf{tl}(\tau)$
  - Since v is the only member of h, h contains only well-typed values, and thus the last value of h has type $\mathsf{hd}(\tau_1)$ in H.
- Suppose $h \neq []$. Suppose all prior values of h are well-typed.
  - By assumption, $S;\ \sigma, x = h :: \mathsf{nil} \vdash e_1 \overset{v}{\hookrightarrow} e_1'$.
  - By assumption and the above, $S;\ \sigma, x = h :: v \vdash e_2 \overset{w}{\hookrightarrow} e_2'$
  - T-LETREC must be the last (non-subtype) rule used in the typing derivation. By inversion:
    * $G;\ H, x : \tau_1 \vdash e_1 : \tau_1$
    * $G;\ H, x : \tau_1 \vdash e_2 : \tau$

- Similarly to before, $G;\ \mathsf{tl}(H) \vdash \mathsf{tl}(\tau_1)$
- By IH and Lemma 3 (same as the $h = []$ case):
  - $G;\ H, x : \tau_1 \vdash v : \mathsf{hd}(\tau_1)$. Thus v is well-typed.
  - $G;\ \mathsf{tl}(H), x : \mathsf{tl}(\tau_1) \vdash e_1' : \mathsf{tl}(\tau_1)$
  - $G;\ H, x : \tau_1 \vdash w : \mathsf{hd}(\tau)$
  - $G;\ \mathsf{tl}(H), x : \mathsf{tl}(\tau_1) \vdash e_2' : \mathsf{tl}(\tau)$
- By (T-FBY), $G;\ \mathsf{tl}(H) \vdash \mathsf{let\ rec}_{h::v}\ x : \mathsf{tl}(\tau_1) = e_1'\ \mathsf{in}\ e_2' : \mathsf{tl}(\tau)$
- Since the only new value added to the history is v and v is well-typed, $h :: v$ remains well-typed. Since it is at the end of h, the last value of h still has type $\mathsf{hd}(\tau_1)$ in H

Case S-LET: Similar to S-LETREC.

Case S-FBY: $e_1\ \mathsf{fby}\ e_2$

- Assume $G;\ H \vdash e_1\ \mathsf{fby}\ e_2 : \{w : b \mid \mathsf{hd}(\psi_1) \wedge \bigcirc\psi_2\}$ and $\sigma \vdash e_1\ \mathsf{fby}\ e_2 \overset{v}{\hookrightarrow} \mathsf{delay}(e_2)$.
- By assumption, $S;\ \sigma \vdash e_1 \overset{v_1}{\hookrightarrow} e_1'$
- T-FBY must be the last (non-subtyping) rule used in the typing derivation. By inversion:
  - $G;\ H \vdash e_1 : \{w : b \mid \mathsf{hd}(\psi_1)\}$
  - $G;\ H \vdash e_2 : \{w : b \mid \psi_2\}$
- By IH, $G;\ H \vdash v_1 : \{w : b \mid \mathsf{hd}(\mathsf{hd}(\psi_1))\} \equiv \{w : b \mid \mathsf{hd}(\psi_1)\}$.
- By applying (T-DELAY) on e_2 and noting that $\mathsf{prev}(\mathsf{tl}(H))(x) \preceq H(x)$ for all $x \in \mathsf{dom}(H)$, $\mathsf{tl}(H) \vdash \mathsf{delay}(e_2) : \{w : b \mid \psi_2\}$

Case S-DELAY: $\mathsf{delay}(e)$

- Assume $G;\ H \vdash \mathsf{delay}(e_1) : \tau$
- By assumption, $S\ \mathsf{prev}(\sigma) \vdash e \overset{v}{\hookrightarrow} e'$
- T-DELAY must be the last (non-subtyping) rule used in the typing derivation. By inversion, $G;\ \mathsf{prev}(H) \vdash e : \tau$
- By IH and using Lemma 3 to split τ:
  - $G;\ \mathsf{prev}(H) \vdash v : \mathsf{hd}(\tau)$. Since v is a value, it is a constant and thus $G;\ H \vdash v : \mathsf{hd}(\tau)$
  - $G;\ \mathsf{tl}(\mathsf{prev}(H)) \vdash e' : \mathsf{tl}(\tau)$
- Since $\mathsf{tl}(\mathsf{prev}(H)) \equiv H$, $G;\ H \vdash e' : \mathsf{tl}(\tau)$
- Using (T-DELAY), $G;\ \mathsf{tl}(H) \vdash \mathsf{delay}(e') : \mathsf{tl}(\tau)$

Case S-IF-T: $\mathsf{if}\ x_c\ \mathsf{then}\ e_t\ \mathsf{else}\ e_e$

- Assume $G;\ H \vdash \mathsf{if}\ x_c\ \mathsf{then}\ e_t\ \mathsf{else}\ e_f : \{w : b \mid \psi\}$ and
  $S;\ \sigma, x = h :: \mathsf{true} \vdash (\mathsf{if}\ x\ \mathsf{then}\ e_t\ \mathsf{else}\ e_f) \overset{v_t}{\hookrightarrow} (\mathsf{if}\ x\ \mathsf{then}\ e_t'\ \mathsf{else}\ e_f')$
- By assumption:
  - $S;\ \sigma, x = h :: \mathsf{true} \vdash e_t \overset{v_t}{\hookrightarrow} e_t'$
  - $S;\ \sigma, x = h :: \mathsf{true} \vdash e_f \overset{v_f}{\hookrightarrow} e_f'$
- T-IF must be the last (non-subtyping) rule used in the typing derivation. By inversion:
  - $G;\ H \vdash x_c : \mathsf{bool}$
  - $G;\ H \vdash e_t : \{w : b \mid \mathsf{impl}(x_c, \psi)\}$
  - $G;\ H \vdash e_f : \{w : b \mid \mathsf{impl}(\neg x_c, \psi)\}$
- By IH and using Lemma 3 to split ψ:
  - $G;\ \mathsf{tl}(H) \vdash e_t' : \mathsf{tl}(\{w : b \mid \mathsf{impl}(x_c, \psi)\})$
  - $G;\ \mathsf{tl}(H) \vdash e_f' : \mathsf{tl}(\{w : b \mid \mathsf{impl}(\neg x_c, \psi)\})$
  - $G;\ \mathsf{tl}(H) \vdash x_c : \mathsf{tl}(\mathsf{bool})$ (equivalent to just (bool))

  - $G;\ H \vdash v_t : \mathsf{hd}(\{w : b \mid \mathsf{impl}(x_c, \psi)\})$. Since the current value of x_c is true, $\mathsf{hd}(\mathsf{impl}(x_c, \psi)) \equiv \mathsf{hd}(\psi)$. Therefore $G;\ H \vdash v_t : \mathsf{hd}(\{w : b \mid \psi\})$.
- By (T-IF): $G;\ \mathsf{tl}(H) \vdash (\text{if } x \text{ then } e'_t \text{ else } e'_f) : \mathsf{tl}(\{w : b \mid \psi\})$

Case S-IF-F: Symmetric to S-IF-T.
Case S-CONST: c where c is a constant

- By Lemma 3, $\mathsf{tl}(\{w : b \mid \Box(w = c)\}) \equiv \{w : b \mid \Box(w = c)\}$
- By Lemma 3, $\{w : b \mid \Box(w = c)\} \preceq \mathsf{hd}(\{w : b \mid \Box(w = c)\}) \equiv \{w : b \mid w = c\}$. Therefore $G;\ H \vdash c : \mathsf{hd}(\{w : b \mid \Box(w = c)\})$
- Since c is a constant, no free variables can occur in c. Therefore, $G;\ \mathsf{tl}(H) \vdash c : \mathsf{tl}(\{w : b \mid \Box(w = c)\})$

Case S-VAR: x

- Assume $G;\ H \vdash x : \{w : b \mid \psi \wedge \Box(w = x)\}$ and $S;\ \sigma, x = h :: v \vdash x \overset{v}{\hookrightarrow} x$
- Since the environment is non-empty (because x can step), this must be a sub-term of either let or let rec. In **Case** S-LETREC, we showed that for $\sigma(x) = h$ and $H(x) = \tau$, the last value of h types to $\mathsf{hd}(\tau)$ in H. Therefore, $G;\ H \vdash v : \mathsf{hd}(\{w : b \mid \psi\})$.
- Because v is the current value of x, $v = x$ (state predicate). Thus the type can be augmented as follows: $G;\ H \vdash v : \mathsf{hd}(\{w : b \mid \psi \wedge \Box(w = x)\})$, noting that for any φ, $\mathsf{hd}(\Box\varphi) \equiv \varphi$.
- By definition of $\mathsf{tl}(H)$, $(\mathsf{tl}(H))(x) = \mathsf{tl}(\{w : b \mid \psi\})$
- By T-VAR, $G;\ \mathsf{tl}(H) \vdash x : \mathsf{tl}(\{w : b \mid \psi \wedge \Box(w = x)\})$

Case S-APP: $f\ y$

- Assume $G, f : x : \tau_1 \to \tau_2;\ H \vdash f\ y : \{w_2 : b_2 \mid \Box\varphi_2[x \mapsto y]\}$ and $S, f(x) = e; \sigma, y = h :: v \vdash f(y) \overset{e[x \mapsto v]}{\hookrightarrow} f(y)$, where $\tau_1 = \{w_1 : b_1 \mid \varphi_1\}$ and $\tau_2 = \{w_2 : b_2 \mid \varphi_2\}$
- The last (non-subtyping) rule in the typing derivation must be T-APP. By inversion, $G, f : x : \tau_1 \to \tau_2;\ H \vdash y : \{w_1 : b_1 \mid \Box\varphi_1\}$
- Because $\mathsf{tl}(\{w_1 : b_1 \mid \Box\varphi_1\}) \equiv \{w_1 : b_1 \mid \Box\varphi_1\}$, then $G, f : x : \tau_1 \to \tau_2;\ \mathsf{tl}(H) \vdash y : \mathsf{tl}(\{w_1 : b_1 \mid \Box\varphi_1\})$
- Using similar reasoning as in **Case** S-VAR, the last value of y's history, v, has type $G, f : x : \tau_1 \to \tau_2;\ H \vdash v : \mathsf{hd}(\{w_1 : b_1 \mid \Box\varphi_1\})$
- Using the standard substitution lemma on values extended to constants, $G, f : x : \tau_1 \to \tau_2;\ H \vdash e[x \mapsto v] : \{w_2 : b_2 \mid \varphi_2[x \mapsto v]\}$. Note that this is equivalent to $\mathsf{hd}(\{w_2 : b_2 \mid \Box\varphi_2[x \mapsto y]\})$ due to the correspondence between v and y.
- By (T-APP) and extending the substitution lemma to type refinements that do not change over time: $G, f : x : \{w_1 : b_1 \mid \varphi_1\} \to \{w_2 : b_2 \mid \varphi_2\};\ \mathsf{tl}(H) \vdash f\ y : \mathsf{tl}(\{w_2 : b_2 \mid \Box\varphi_2[x \mapsto y]\})$ which is equivalent to $\{w_2 : b_2 \mid \Box\varphi_2[x \mapsto y]\}$

Case S-MODELS: e models r

- Assume $S, \sigma \vdash e \text{ models } r \overset{v}{\hookrightarrow} e' \text{ models } r$. Also assume that the program is not running in robot-mode (ie the program behavior is exclusively defined by the model e. Further assume that $G;\ H \vdash e \text{ models } r : \tau$.
- By assumption, $S, \sigma \vdash e \overset{v}{\hookrightarrow} e'$.
- T-MODELS must be the last (non-subtype) rule used in the typing derivation. By inversion, $G;\ H \vdash e : \tau$.
- By IH, $G;\ \mathsf{tl}(H) \vdash e' : \mathsf{tl}(\tau)$
- By T-MODELS, $G;\ \mathsf{tl}(H) \vdash e' \text{ models } r : \mathsf{tl}(\tau)$

□

B Collision Avoidance Source Code

```
let vfi:{v: float | v > 0.} = 0.5
let dt:{v: float | v > 0.} = 0.1
let b:{v: float | v > 0.} = 0.136
let xli:{v: float | v > 0.} = 2.
let xbrake:{v: float | v >= xli} = 4.
let amax:{v: float | v >= 0.} = 0.06

let node follower_control ((xf, vf, xl, vl):
  {(vxf:float)*(vvf:float)*(vxl:float)*(vvl:float) | true}) :
  {v:float | ((((xf +. ((vf*.vf)/.(2.*.b)) +.
    ((1.+.(amax/.b))*.(2.*.dt*.vf+.((amax*.(4.*.dt*.dt))/. 2.))) +.
    ((vf*.dt)/. 2.)) +. 0.5
    <
    (xl +. (((vl-.(b*.dt))*.(vl-.(b*.dt)))/.(2.*.b)) +.
    (((vl-.(b*.dt))*.dt)/. 2.))) && (v = amax)) ||
    (((xf +. ((vf*.vf)/.(2.*.b)) +.
    ((1.+.(amax /. b))*.(2.*.dt*.vf+.((amax*.(4.*.dt*.dt))/. 2.))) +.
    ((vf*.dt)/. 2.)) +. 0.5
    >=
    (xl +. (((vl-.(b*.dt))*.(vl-.(b*.dt)))/.(2.*.b)) +.
    (((vl-.(b*.dt))*.dt)/. 2.))) && (v = -.b)))} =
  (if ((xf +. ((vf*.vf)/.(2.*.b)) +.
    ((1.+.(amax /. b))*.(2.*.dt*.vf+.((amax*.(4.*.dt*.dt))/. 2.)))
    +.
    ((vf*.dt)/. 2.)) +. 0.5
    >=
    (xl +. (((vl-.(b*.dt))*.(vl-.(b*.dt)))/.(2.*.b)) +.
    (((vl-.(b*.dt))*.dt)/. 2.)))
  then (-.b)
  else amax)

let node max (x:{v:float | true}) :
  {v:float | ( x >= 0. && v = x) || (x < 0. && v = 0)} =
  (if (x < 0.) then 0. else x)

let node leader_control (xl:{v:float | true}) :
  {v:float | ((xl >= xbrake) && (v = -.b)) ||
    ((xl < xbrake) && (v = 0.))} =
  (if (xl >= xbrake) then (-.b) else 0.)

let node exec () :
  {(vxf:float)*(vvf:float)*(vaf:float)*
   (vxl:float)*(vvl:float)*(vval:float) | true } =
  let rec ((xf, vf, af, xl, vl, al):
    {(x:float)*(v:float)*(a:float)*(x2:float)*(v2:float)*(a2:float) |
      (x2 > x) && (x2 >= xli) && (v2 >= 0.) && (v >= 0.) &&
```

```
47        (x >= 0.) && (a >= -.b) &&
48          ((((x +. ((v*.v)/.(2.*.b)) +.
49            ((1.+.(amax /. b))*.
50              (2.*.dt*.v+.((amax*.(4.*.dt*.dt))/. 2.))) +.
51                ((v*.dt)/. 2.)) +. 0.5 <
52            (x2 +. (((v2-.(b*.dt))*.(v2-.(b*.dt)))/.(2.*.b)) +.
53              (((v2-.(b*.dt))*.dt)/. 2.))) && (a = amax)) ||
54          (((x +. ((v*.v)/.(2.*.b)) +.
55            ((1.+.(amax /. b))*.
56              (2.*.dt*.v+.((amax*.(4.*.dt*.dt))/. 2.)))  +.
57                ((v*.dt)/. 2.)) +. 0.5 >=
58            (x2 +. (((v2-.(b*.dt))*.(v2-.(b*.dt)))/.(2.*.b)) +.
59              (((v2-.(b*.dt))*.dt)/. 2.))) && (a = -.b))) &&
60          (((x2 >= xbrake) && (a2 = -.b)) ||
61           ((x2 < xbrake) && (a2 = 0.))) &&
62        ((x +. ((v*.v)/.(2.*.b)) +. ((v*.dt)/. 2.)) <
63        (x2 +. ((v2*.v2)/.(2.*.b)) +. ((v2*.dt)/. 2.))})) =
64      (0., vfi, amax, xli, vfi, 0.) fby
65      (let vl_next = (max(vl +. (al*.dt)))
66        models (robot_get ("leader_vel")) in
67      (let xl_next = (xl +. (vl_next *. dt))
68        models (robot_get ("leader_x")) in
69      (let vf_next = (max(vf +. (af*.dt)))
70        models (robot_get ("follower_vel")) in
71      (let xf_next = (xf +. (vf_next *. dt))
72        models (robot_get ("follower_x")) in
73      (xf_next, vf_next,
74      ((if ((xf_next+.((vf_next*.vf_next)/.(2.*.b))+.
75        ((1.+.(amax /. b))*.
76        (2.*.dt*.vf_next+.((amax*.(4.*.dt*.dt))/. 2.)))+.
77        ((vf_next*.dt)/.2.))+.0.5
78        >=
79        (xl_next+.(((vl_next-.(b*.dt))*.(vl_next-.(b*.dt)))/.(2.*.b)) +.
80        (((vl_next-.(b*.dt))*.dt)/. 2.)))
81        then (-.b)
82        else amax)),
83         xl_next, vl_next,
84         ((if (xl_next >= xbrake) then (-.b) else 0.)))))))
85      in  (() models robot_str ("accel", af)); (xf, vf, af, xl, vl, al)
86
87  let hybrid main () =
88    let rec trigger = (period (dt)) in
89      present (trigger) -> (let (x, v, a, xl, vl, al) = exec () in
90      (print_float (Timestamp.gettimeofday ());
91      print_string ","; print_float x; print_string ","; print_float v;
92      print_string ","; print_float a; print_string ","; print_float xl;
93      print_string ","; print_float vl; print_string ",";
94      print_float al; print_newline ())) else ()
```

References

[1] Rajeev Alur, Costas Courcoubetis, Thomas A. Henzinger, and Pei Hsin Ho. 1993. Hybrid automata: An algorithmic approach to the specification and verification of hybrid systems. In *Hybrid Systems*, G. Goos, J. Hartmanis, Robert L. Grossman, Anil Nerode, Anders P. Ravn, and Hans Rischel (Eds.). Vol. 736. Springer, Berlin, Heidelberg, 209–229. https://doi.org/10.1007/3-540-57318-6_30

[2] Aaron D. Ames, Jessy W. Grizzle, and Paulo Tabuada. 2014. Control barrier function based quadratic programs with application to adaptive cruise control. In *53rd IEEE Conference on Decision and Control (CDC 2014)*. IEEE, Los Angeles, CA, USA, 6271–6278. https://doi.org/10.1109/CDC.2014.7040372

[3] Abhishek Anand and Ross Knepper. 2015. ROSCoq: Robots Powered by Constructive Reals. In *Interactive Theorem Proving (ITP 2015)*, Christian Urban and Xingyuan Zhang (Eds.). Vol. 9236. Springer International Publishing, Cham, 34–50. https://doi.org/10.1007/978-3-319-22102-1_3

[4] Guillaume Baudart, Louis Mandel, Eric Atkinson, Benjamin Sherman, Marc Pouzet, and Michael Carbin. 2020. Reactive probabilistic programming. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2020)*. Association for Computing Machinery, New York, NY, USA, 898–912. https://doi.org/10.1145/3385412.3386009

[5] Jan Baumeister, Bernd Finkbeiner, Sebastian Schirmer, Maximilian Schwenger, and Christoph Torens. 2020. RTLola Cleared for Take-Off: Monitoring Autonomous Aircraft. In *Computer Aided Verification (CAV 2020)*, Shuvendu K. Lahiri and Chao Wang (Eds.). Vol. 12225. Springer International Publishing, Cham, 28–39. https://doi.org/10.1007/978-3-030-53291-8_3 Series Title: Lecture Notes in Computer Science.

[6] Albert Benveniste, Timothy Bourke, Benoit Caillaud, Bruno Pagano, and Marc Pouzet. 2014. A type-based analysis of causality loops in hybrid systems modelers. In *Proceedings of the 17th international conference on Hybrid systems: computation and control (HSCC 2014)*. ACM Press, Berlin, Germany, 71–82. https://doi.org/10.1145/2562059.2562125

[7] Albert Benveniste, Timothy Bourke, Benoît Caillaud, and Marc Pouzet. 2011. A hybrid synchronous language with hierarchical automata: static typing and translation to synchronous code. In *Proceedings of the 9th ACM international conference on Embedded software (EMSOFT 2011)*. ACM, Taipei, Taiwan, 137–148. https://doi.org/10.1145/2038642.2038664

[8] Rose Bohrer, Yong Kiam Tan, Stefan Mitsch, Magnus O. Myreen, and André Platzer. 2018. VeriPhy: verified controller executables from verified cyber-physical system models. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2018)*. ACM, Philadelphia PA USA, 617–630. https://doi.org/10.1145/3192366.3192406

[9] Michael H Borkowski, Niki Vazou, and Ranjit Jhala. 2024. Mechanizing refinement types. *Proceedings of the ACM on Programming Languages (POPL 2024)* 8 (2024), 2099–2128. https://doi.org/10.1145/3632912

[10] Sylvain Boulmé and Grégoire Hamon. 2001. Certifying Synchrony for Free. In *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR 2001)*, G. Goos, J. Hartmanis, J. Van Leeuwen, Robert Nieuwenhuis, and Andrei Voronkov (Eds.). Vol. 2250. Springer, Berlin, Heidelberg, 495–506. https://doi.org/10.1007/3-540-45653-8_34

[11] Timothy Bourke, Lélio Brun, Pierre-Evariste Dagand, Xavier Leroy, Marc Pouzet, and Lionel Rieg. 2017. A Formally Verified Compiler for Lustre. *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)* 52, 6 (2017), 586–601. https://doi.org/10.1145/3062341.3062358

[12] Timothy Bourke and Marc Pouzet. 2013. Zélus: a synchronous language with ODEs. In *Proceedings of the 16th international conference on Hybrid systems: computation and control (HSCC 2013)*. ACM Press, Philadelphia, Pennsylvania, USA, 113. https://doi.org/10.1145/2461328.2461348

[13] F. Boussinot and R. de Simone. 1991. The ESTEREL language. *Proc. IEEE* 79, 9 (Sept. 1991), 1293–1304. https://doi.org/10.1109/5.97299

[14] P. Caspi, D. Pilaud, N. Halbwachs, and J. A. Plaice. 1987. LUSTRE: A Declarative Language for Real-Time Programming. In *Proceedings of the 14th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL 1987)* (Munich, West Germany). ACM, New York, NY, USA, 178–188. https://doi.org/10.1145/41625.41641

[15] Paul Caspi and Marc Pouzet. 1996. Synchronous Kahn networks. In *Proceedings of the first ACM SIGPLAN international conference on Functional programming (ICFP 1996)*. ACM Press, Philadelphia, Pennsylvania, United States, 226–238. https://doi.org/10.1145/232627.232651

[16] Paul Caspi and Marc Pouzet. 1998. A Co-iterative Characterization of Synchronous Stream Functions. *Electronic Notes in Theoretical Computer Science* 11 (1998), 1–21. https://doi.org/10.1016/S1571-0661(04)00050-7

[17] Adrien Champion, Alain Mebsout, Christoph Sticksel, and Cesare Tinelli. 2016. The Kind 2 Model Checker. In *Computer Aided Verification (CAV 2016)*, Swarat Chaudhuri and Azadeh Farzan (Eds.). Springer International Publishing, Cham, 510–517. https://doi.org/10.1007/978-3-319-41540-6_29

[18] Jiawei Chen, José Luiz Vargas de Mendonça, Bereket Shimels Ayele, Bereket Ngussie Bekele, Shayan Jalili, Pranjal Sharma, Nicholas Wohlfeil, Yicheng Zhang, and Jean-Baptiste Jeannin. 2024. Synchronous Programming with Refinement Types. https://doi.org/10.48550/arXiv.2406.06221 arXiv:2406.06221 [cs]. Extended version.

[19] Jiawei Chen, José Luiz Vargas de Mendonça, Bereket Shimels Ayele, Bereket Ngussie Bekele, Shayan Jalili, Pranjal Sharma, Nicholas Wohlfeil, Yicheng Zhang, and Jean-Baptiste Jeannin. 2024. Synchronous Programming with Refinement Types. https://doi.org/10.5281/zenodo.12702677 Artifact.

[20] Jiawei Chen, José Luiz Vargas de Mendonça, Shayan Jalili, Bereket Ayele, Bereket Ngussie Bekele, Zhemin Qu, Pranjal Sharma, Tigist Shiferaw, Yicheng Zhang, and Jean-Baptiste Jeannin. 2022. Synchronous Programming and Refinement Types in Robotics: From Verification to Implementation. In *Proceedings of the 8th ACM SIGPLAN International Workshop on Formal Techniques for Safety-Critical Systems (FTSCS 2022)*. ACM, New York, NY, USA, 68–79. https://doi.org/10.1145/3563822.3568015

[21] Jean-Louis Colaço, Grégoire Hamon, and Marc Pouzet. 2006. Mixing Signals and Modes in Synchronous Data-Flow Systems. In *Proceedings of the 6th ACM & IEEE International Conference on Embedded Software (EMSOFT 2006)* (Seoul, Korea). ACM, New York, NY, USA, 73–82. https://doi.org/10.1145/1176887.1176899

[22] Jean-Louis Colaço, Bruno Pagano, and Marc Pouzet. 2017. SCADE 6: A formal language for embedded critical software development (invited paper). In *2017 International Symposium on Theoretical Aspects of Software Engineering (TASE 2017)*. IEEE, 1–11. https://doi.org/10.1109/TASE.2017.8285623

[23] Pascal Cuoq and Marc Pouzet. 2001. Modular causality in a synchronous stream language. In *European Symposium on Programming (ESOP 2001)*, Gerhard Goos, Juris Hartmanis, Jan van Leeuwen, and David Sands (Eds.), Vol. 2028. Springer, Berlin, Heidelberg, 237–251. https://doi.org/10.1007/3-540-45309-1_16 Series Title: Lecture Notes in Computer Science.

[24] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008)*, David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Doug Tygar, Moshe Y. Vardi, Gerhard Weikum, C. R. Ramakrishnan, and Jakob Rehof (Eds.). Vol. 4963. Springer, Berlin, Heidelberg, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24

[25] Matthias Eichholz, Eric Hayden Campbell, Matthias Krebs, Nate Foster, and Mira Mezini. 2022. Dependently-typed data plane programming. *Proceedings of the ACM on Programming Languages* 6, POPL 2022 (2022), 1–28. https://doi.org/10.1145/3498701

[26] Georgios E Fainekos and George J Pappas. 2009. Robustness of temporal logic specifications for continuous-time signals. *Theoretical Computer Science* 410, 42 (2009), 30. https://doi.org/10.1016/j.tcs.2009.06.021

[27] Spencer P. Florence, Shu-Hung You, Jesse A. Tov, and Robert Bruce Findler. 2019. A Calculus for Esterel: If Can, Can. If No Can, No Can. *Proceedings of the ACM on Programming Languages* 3, POPL 2019 (2019), 1–29. https://doi.org/10.1145/3290374

[28] Khalil Ghorbal and André Platzer. 2014. Characterizing Algebraic Invariants by Differential Radical Invariants. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2014)*, Erika Ábrahám and Klaus Havelund (Eds.). Vol. 8413. Springer, Berlin, Heidelberg, 279–294. https://doi.org/10.1007/978-3-642-54862-8_19 Series Title: Lecture Notes in Computer Science.

[29] Ritwika Ghosh, Chiao Hsieh, Sasa Misailovic, and Sayan Mitra. 2020. Koord: A Language for Programming and Verifying Distributed Robotics Application. *Proceedings of the ACM on Programming Languages* 4, OOPSLA 2020, Article 232 (2020), 30 pages. https://doi.org/10.1145/3428300

[30] Ritwika Ghosh, Joao P. Jansch-Porto, Chiao Hsieh, Amelia Gosse, Minghao Jiang, Hebron Taylor, Peter Du, Sayan Mitra, and Geir Dullerud. 2020. CyPhyHouse: A programming, simulation, and deployment toolchain for heterogeneous distributed coordination. In *IEEE International Conference on Robotics and Automation (ICRA 2020)*. IEEE, 6654–6660. https://doi.org/10.1109/ICRA40945.2020.9196513

[31] George Hagen and Cesare Tinelli. 2008. Scaling Up the Formal Verification of Lustre Programs with SMT-Based Techniques. In *Formal Methods in Computer-Aided Design (FMCAD 2008)*. IEEE, 1–9. https://doi.org/10.1109/FMCAD.2008.ECP.19

[32] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud. 1991. The synchronous data flow programming language LUSTRE. *Proc. IEEE* 79, 9 (Sept. 1991), 1305–1320. https://doi.org/10.1109/5.97300

[33] Jean-Baptiste Jeannin and André Platzer. 2014. dTL^2: Differential Temporal Dynamic Logic with Nested Temporalities for Hybrid Systems. In *Automated Reasoning: 7th International Joint Conference (IJCAR 2014), Held as Part of the Vienna Summer of Logic (VSL 2014)*, Stéphane Demri, Deepak Kapur, and Christoph Weidenbach (Eds.). Vol. 8562. Springer International Publishing, Cham, 292–306. https://doi.org/10.1007/978-3-319-08587-6_22 Series Title: Lecture Notes in Computer Science.

[34] Ranjit Jhala and Niki Vazou. 2021. Refinement types: A tutorial. *Foundations and Trends® in Programming Languages* 6, 3–4 (2021), 159–317. https://doi.org/10.1561/2500000032

[35] Eduard Kamburjan. 2021. From post-conditions to post-region invariants: deductive verification of hybrid objects. In *Proceedings of the 24th International Conference on Hybrid Systems: Computation and Control (HSCC 2021)*. ACM, Nashville Tennessee, 1–11. https://doi.org/10.1145/3447928.3456633

[36] Sarah M. Loos, André Platzer, and Ligia Nistor. 2011. Adaptive Cruise Control: Hybrid, Distributed, and Now Formally Verified. In *Formal Methods (FM 2011)*, Michael Butler and Wolfram Schulte (Eds.). Vol. 6664. Springer, Berlin, Heidelberg, 42–56. https://doi.org/10.1007/978-3-642-21437-0_6 Series Title: Lecture Notes in Computer Science.

[37] Gregory Malecha, Daniel Ricketts, Mario M. Alvarez, and Sorin Lerner. 2016. Towards foundational verification of cyber-physical systems. In *Science of Security for Cyber-Physical Systems Workshop (SOSCYPS 2016)*. IEEE, 1–5. https://doi.org/10.1109/SOSCYPS.2016.7580000

[38] Zohar Manna and Amir Pnueli. 1992. *The Temporal Logic of Reactive and Concurrent Systems*. Springer New York, New York, NY. https://doi.org/10.1007/978-1-4612-0931-7

[39] Aakar Mehra, Wen-Loong Ma, Forrest Berg, Paulo Tabuada, Jessy W. Grizzle, and Aaron D. Ames. 2015. Adaptive cruise control: Experimental validation of advanced controllers on scale-model cars. In *American Control Conference (ACC 2015)*. IEEE, Chicago, IL, USA, 1411–1418. https://doi.org/10.1109/ACC.2015.7170931

[40] Xinming Ou, Gang Tan, Yitzhak Mandelbaum, and David Walker. 2004. Dynamic typing with dependent types. In *Exploring New Frontiers of Theoretical Informatics: IFIP 18th World Computer Congress TC1 3rd International Conference on Theoretical Computer Science (TCS 2004)* (Toulouse, France). Springer, Boston, MA, 437–450. https://doi.org/10.1007/1-4020-8141-3_34

[41] Ioannis Parissis. 1996. *Test de logiciels synchrones spécifiés en Lustre*. Ph. D. Dissertation. Université Joseph-Fourier-Grenoble I. https://theses.hal.science/tel-00005010v1/document

[42] Benjamin C. Pierce. 2005. *Advanced topics in types and programming languages*. MIT press. https://doi.org/10.1093/comjnl/bxh138

[43] André Platzer. 2008. Differential Dynamic Logic for Hybrid Systems. *Journal of Automated Reasoning* 41, 2 (Aug. 2008), 143–189. https://doi.org/10.1007/s10817-008-9103-8

[44] André Platzer. 2018. *Logical Foundations of Cyber-Physical Systems*. Springer International Publishing, Cham. https://doi.org/10.1007/978-3-319-63588-0

[45] Amir Pnueli. 1977. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science (SFCS 1977)*. IEEE, 46–57. https://doi.org/10.1109/SFCS.1977.32 ISSN: 0272-5428.

[46] Stephen Prajna and Ali Jadbabaie. 2004. Safety Verification of Hybrid Systems Using Barrier Certificates. In *Hybrid Systems: Computation and Control (HSCC 2004)*, Rajeev Alur and George J. Pappas (Eds.). Springer, Berlin, Heidelberg, 477–492. https://doi.org/10.1007/978-3-540-24743-2_32

[47] Christophe Ratel. 1992. *Définition et Réalisation d'un Outil de Vérification Formelle de Programmes LUSTRE: le Système LESAR*. Ph. D. Dissertation. Université Joseph Fourier. https://tel.archives-ouvertes.fr/file/index/docid/341223/filename/Ratel.Christophe_1992_these.pdf

[48] Patrick M. Rondon, Ming Kawaguci, and Ranjit Jhala. 2008. Liquid types. In *Proceedings of the 2008 ACM SIGPLAN conference on Programming language design and implementation (PLDI 2008)*. ACM Press, Tucson, AZ, USA, 159. https://doi.org/10.1145/1375581.1375602

[49] Astrid Rupp, Markus Tranninger, Raffael Wallner, Jasmina Zubača, Martin Steinberger, and Martin Horn. 2019. Fast and low-cost testing of advanced driver assistance systems using small-scale vehicles. *IFAC-PapersOnLine* 52, 5 (2019), 34–39. https://doi.org/10.1016/j.ifacol.2019.09.006

[50] CG Rusu, IT Birou, and E Szöke. 2010. Fuzzy based obstacle avoidance system for autonomous mobile robot. In *IEEE International Conference on Automation, Quality and Testing, Robotics (AQTR 2010)*, Vol. 1. IEEE, 1–6. https://doi.org/10.1109/AQTR.2010.5520862

[51] Nikhil Swamy, Cătălin Hriţcu, Chantal Keller, Aseem Rastogi, Antoine Delignat-Lavaud, Simon Forest, Karthikeyan Bhargavan, Cédric Fournet, Pierre-Yves Strub, Markulf Kohlweiss, Jean-Karim Zinzindohoue, and Santiago Zanella-Béguelin. 2016. Dependent Types and Multi-Monadic Effects in F*. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2016)* (St. Petersburg, FL, USA). ACM, New York, NY, USA, 256–270. https://doi.org/10.1145/2837614.2837655

[52] Alfred Tarski. 1951. A Decision Method for Elementary Algebra and Geometry. In *Quantifier Elimination and Cylindrical Algebraic Decomposition*. Springer, 24–84. https://doi.org/10.1007/978-3-7091-9459-1_3

[53] Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. 2014. Refinement types for Haskell. In *Proceedings of the 19th ACM SIGPLAN international conference on Functional programming (ICFP 2014)*. ACM, New York, NY, USA, 269–282. https://doi.org/10.1145/2628136.2628161

[54] Shuhuan Wen, Wei Zheng, Jinghai Zhu, Xiaoli Li, and Shengyong Chen. 2011. Elman fuzzy adaptive control for obstacle avoidance of mobile robots using hybrid force/position incorporation. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 42, 4 (2011), 603–608. https://doi.org/10.1109/TSMCC.2011.2157682

[55] A.K. Wright and M. Felleisen. 1994. A Syntactic Approach to Type Soundness. *Information and Computation* 115, 1 (Nov. 1994), 38–94. https://doi.org/10.1006/inco.1994.1093

Received 2024-02-28; accepted 2024-06-18